ID: 169251

Desenvolvimento e Implementação de um Processador MIPS 32 em FPGA

São José dos Campos - Brasil

Julho de 2025

ID: 169251

Desenvolvimento e Implementação de um Processador MIPS 32 em FPGA

Relatório apresentado à Universidade Federal de São Paulo como parte dos requisitos para aprovação na disciplina de Laboratório de Sistemas Computacionais: Arquitetura e Organização de Computadores.

Docente: Prof. Dr. Tiago de Oliveira

Universidade Federal de São Paulo - UNIFESP

Instituto de Ciência e Tecnologia - Campus São José dos Campos

São José dos Campos - Brasil

Julho de 2025

Resumo

Este relatório apresenta o planejamento teórico e a implementação, em linguagem de descrição de *hardware*, de um processador baseado na arquitetura MIPS 32, com foco na aplicação em um kit FPGA. São abordados o conjunto de instruções adotado, os formatos das instruções, a estrutura do caminho de dados (*datapath*) e os sinais de controle necessários para a execução das operações. A partir do projeto arquitetural definido, desenvolveu-se em SystemVerilog o processador de maneira funcional, permitindo a execução de programas a partir do conjunto de instruções, sendo capaz de executar algoritmos de lógica de programação.

Palavras-chaves: Arquitetura e Organização de Computadores. Verilog. FPGA. Processadores. MIPS.

Lista de ilustrações

Figura 1 – Kit FPGA Altera DE2-115 11
Figura 2 – Caminho de dados básico para uma arquitetura MIPS 12
Figura 3 – Modos de endereçamento 14
Figura 4 – Modelo esquemático da memória 17
Figura 5 – Modelo esquemático de um multiplexador 19
Figura 6 – Modelo esquemático da ULA 20
Figura 7 – *Datapath* do processador 26
Figura 8 – Simulação do *Program Counter* com sinais de controle inativos 44
Figura 9 – Simulação do *Program Counter* com sinal `Branch` ativo 44
Figura 10 – Simulação do *Program Counter* com sinal `Jump` ativo 45
Figura 11 – Simulação do *Program Counter* com sinal `JumpReg` ativo 45
Figura 12 – Simulação do *Program Counter* com sinal `Halt` ativo 45
Figura 13 – Simulação da memória de instruções 46
Figura 14 – Simulação do extensor de sinal 46
Figura 15 – Simulação do banco de registradores 46
Figura 16 – Simulação do banco de registradores 47
Figura 17 – Simulação da ALU para operações aritméticas 48
Figura 18 – Simulação da ALU para operações lógicas 48
Figura 19 – Simulação da ALU para operações de deslocamento 48
Figura 20 – Simulação do controle da ALU 49
Figura 21 – Simulação da Unidade Lógica de *Branch* 49
Figura 22 – Simulação da Memória de Dados 50
Figura 23 – Simulação da seleção de dados de escrita/saída 51
Figura 24 – Funcionamento do algoritmo - Entrada do total de inserções 53
Figura 25 – Funcionamento do algoritmo - Primeira inserção 53
Figura 26 – Funcionamento do algoritmo - Segunda inserção 54
Figura 27 – Funcionamento do algoritmo - Terceira inserção 54
Figura 28 – Funcionamento do algoritmo - Quarta inserção 55
Figura 29 – Funcionamento do algoritmo - Quinta inserção 55
Figura 30 – Funcionamento do algoritmo - *Loop* de execução 56
Figura 31 – Funcionamento do algoritmo - Impressão do maior valor 56
Figura 32 – Funcionamento do algoritmo - Impressão do menor valor 57
Figura 33 – Funcionamento do algoritmo - Instrução de Halt 57

Lista de tabelas

Tabela 1 – Formato de instruções do tipo R 23
Tabela 2 – Formato de instruções do tipo I 23
Tabela 3 – Formato de instruções do tipo J 23
Tabela 4 – Conjunto de instruções 24
Tabela 5 – Instruções de formato R com seus valores de opcode e funct 25
Tabela 6 – Instruções de formato I com seus valores de opcode e funct 25
Tabela 7 – Instruções de formato J com seus valores de opcode e funct 25
Tabela 8 – Descrição dos sinais de controle 27
Tabela 9 – Sinais de controle por instrução 27
Tabela 10 – Sinais de operação da ALU 28

Sumário

1 INTRODUÇÃO 8

2 OBJETIVOS 9
2.1 Geral 9
2.2 Específicos 9

3 FUNDAMENTAÇÃO TEÓRICA 10
3.1 Linguagens de descrição de *hardware* (HDL) 10
3.1.1 Verilog 10
3.2 FPGA 10
3.3 Arquitetura de processadores 11
3.3.1 Arquiteturas CISC e RISC 11
3.3.2 Arquiteturas de Von Neumann e Harvard 12
3.3.3 Arquitetura MIPS 12
3.4 Instrução de máquina 13
3.5 Conjunto de instruções 13
3.5.1 Formato de instruções 13
3.6 Modos de Endereçamento 14
3.6.1 Endereçamento imediato 14
3.6.2 Endereçamento em registrador 15
3.6.3 Endereçamento de base ou deslocamento 15
3.6.4 Endereçamento relativo ao PC 15
3.6.5 Endereçamento pseudodireto 15
3.7 Circuitos Combinacionais 16
3.8 Circuitos Sequenciais 16
3.9 Memória 17
3.9.1 Memórias RAM 17
3.9.2 Memórias ROM 18
3.10 Registradores 18
3.10.1 *Program Counter (PC)* 18
3.11 Multiplexadores (MUX) 18
3.12 Unidade de Controle (UC) 19
3.13 Unidade Lógica Aritmética (ALU) 19

4 DESENVOLVIMENTO 21
4.1 Características do processador 21

4.2 Conjunto de Instruções . . . 22
4.3 *Datapath* . . . 26
4.4 Implementação do Processador . . . 28
4.4.1 Processador . . . 29
4.4.2 *Program Counter* . . . 30
4.4.3 Memória de Instrução . . . 30
4.4.4 Unidade de controle . . . 31
4.4.5 Extensor de sinal . . . 31
4.4.6 Banco de registradores . . . 32
4.4.7 Unidade Lógica e Aritmética . . . 33
4.4.7.1 Unidade de Controle da ALU . . . 34
4.4.8 Unidade Lógica de *Branch* . . . 34
4.4.9 Memória de dados . . . 35
4.4.10 Seleção de dados de escrita/saída . . . 36
4.4.11 Módulo de Atualização do *Program Counter* . . . 36
4.4.12 Topo do sistema . . . 37
4.4.13 Divisor de Clock . . . 38
4.4.14 Debouncer . . . 38
4.4.15 Saída para Display de Sete Segmentos . . . 39
4.5 Pinagem do kit FPGA . . . 40
4.6 Algoritmo de validação . . . 40

5 RESULTADOS OBTIDOS E DISCUSSÕES . . . 43
5.1 Simulação dos módulos da Unidade de Processamento . . . 43
5.1.1 Simulando o *Program Counter* . . . 43
5.1.2 Simulando a Memória de Instrução . . . 45
5.1.3 Simulando o Extensor de Sinal . . . 46
5.1.4 Simulando o Banco de Registradores . . . 46
5.1.5 Multiplexador para seleção de registrador de escrita . . . 47
5.1.6 Simulando a Unidade Lógica e Aritmética . . . 47
5.1.7 Unidade de Controle da ALU . . . 48
5.1.8 Simulando a Unidade Lógica de *Branch* . . . 49
5.1.9 Simulando a Memória de Dados . . . 50
5.1.10 Simulando a seleção de dados de escrita/saída . . . 51
5.2 Verificação do sistema computacional com algoritmo de lógica de programação . . . 51

6 CONSIDERAÇÕES FINAIS . . . 58

REFERÊNCIAS 59

APÊNDICES 61

APÊNDICE A – SYSTEM_TOP.SV 62

APÊNDICE B – MIPS_PROCESSOR.SV 64

APÊNDICE C – PROGRAM_COUNTER.SV 68

APÊNDICE D – PC_UPDATE_LOGIC.SV 69

APÊNDICE E – INSTRUCTION_ROM.SV 71

APÊNDICE F – CONTROL_UNIT.SV 72

APÊNDICE G – SIGN_EXTENSION.SV 82

APÊNDICE H – REGISTER_FILE.SV 83

APÊNDICE I – MUX_WRITE_REG.SV 84

APÊNDICE J – ALU.SV 85

APÊNDICE K – MUX_2TO1.SV 87

APÊNDICE L – ALU_CONTROL.SV 88

APÊNDICE M – BRANCH_LOGIC_UNIT.SV 90

APÊNDICE N – DATA_RAM.SV 91

APÊNDICE O – MUX_WRITE_BACK.SV 92

APÊNDICE P – CLOCK_DIVIDER.SV 93

APÊNDICE Q – BUTTON_DEBOUNCER.SV 94

APÊNDICE R – CLOCK_ENABLE.SV 95

APÊNDICE S – DEBOUNCE_DFF_EN.SV 96

APÊNDICE T – DISPLAY_OUTPUT.SV 97

APÊNDICE U – BIN_TO_BCD.SV 99

1 Introdução

A evolução tecnológica dos últimos anos tem transformado profundamente a forma como dispositivos computacionais são concebidos, utilizados e integrados à vida cotidiana. Computadores, celulares, sistemas embarcados passaram a desempenhar funções cada vez mais complexas, exigindo arquiteturas internas mais eficientes e adaptáveis. Com isso, compreender os fundamentos que regem a estrutura e o funcionamento dessas máquinas torna-se essencial para profissionais da área de computação.

No campo da Engenharia da Computação, a disciplina de Arquitetura e Organização de Computadores desempenha papel central ao introduzir os elementos que definem tanto a forma como o sistema opera internamente quanto como ele é percebido pelo programador. Segundo Stallings (2010)(1), enquanto a arquitetura aborda os aspectos visíveis ao programador — como o conjunto de instruções, modos de endereçamento e registradores —, a organização refere-se à implementação desses aspectos por meio de circuitos, sinais de controle e unidades funcionais.

Esse conhecimento é fundamental para a compreensão do funcionamento dos processadores, que são os componentes responsáveis por interpretar e executar instruções em qualquer sistema computacional. Estudar como uma instrução é buscada na memória, decodificada, processada e, por fim, como os dados são manipulados e armazenados, permite ao futuro engenheiro projetar e analisar sistemas de forma crítica e fundamentada.

Diante disso, o presente relatório propõe-se a explorar os conceitos fundamentais envolvidos no desenvolvimento de um processador, desde seu planejamento até a implementação final. Serão abordadas etapas como a definição do conjunto de instruções, a estruturação do caminho de dados e a implementação completa do processador em linguagem de descrição de *hardware*, com foco na aplicação prática e funcional em um kit FPGA. Essa abordagem visa não apenas consolidar o conhecimento teórico, mas também demonstrar sua aplicação prática por meio do desenvolvimento de um sistema computacional coerente com os princípios da engenharia e capaz de atender a requisitos reais de execução.

2 Objetivos

2.1 Geral

Projetar e implementar uma arquitetura de processador baseada na arquitetura MIPS, utilizando a linguagem de descrição de hardware Verilog e um kit FPGA. O sistema será composto por um conjunto definido de instruções, memória e interface de entrada e saída.

2.2 Específicos

- Projetar uma arquitetura baseada na MIPS 32, com definição do conjunto e formato de instruções, modos de endereçamento e caminho de dados;
- Desenvolver e integrar a unidade de processamento, unidade de controle e a interface de entrada e saída com o kit FPGA;
- Implementar um algoritmo de lógica de programação utilizando o conjunto de instruções definido;
- Validar o funcionamento do sistema computacional por meio da sua síntese e operação em um kit FPGA.

3 Fundamentação Teórica

3.1 Linguagens de descrição de *hardware* (HDL)

As linguagens de descrição de hardware, conhecidas como HDL (*Hardware Description Languages*), foram criadas para representar o funcionamento e a estrutura de sistemas digitais de forma textual e abstrata. Elas permitem descrever circuitos por meio de construções lógicas e comportamentais, facilitando tanto o projeto quanto a simulação de sistemas complexos (2).

3.1.1 Verilog

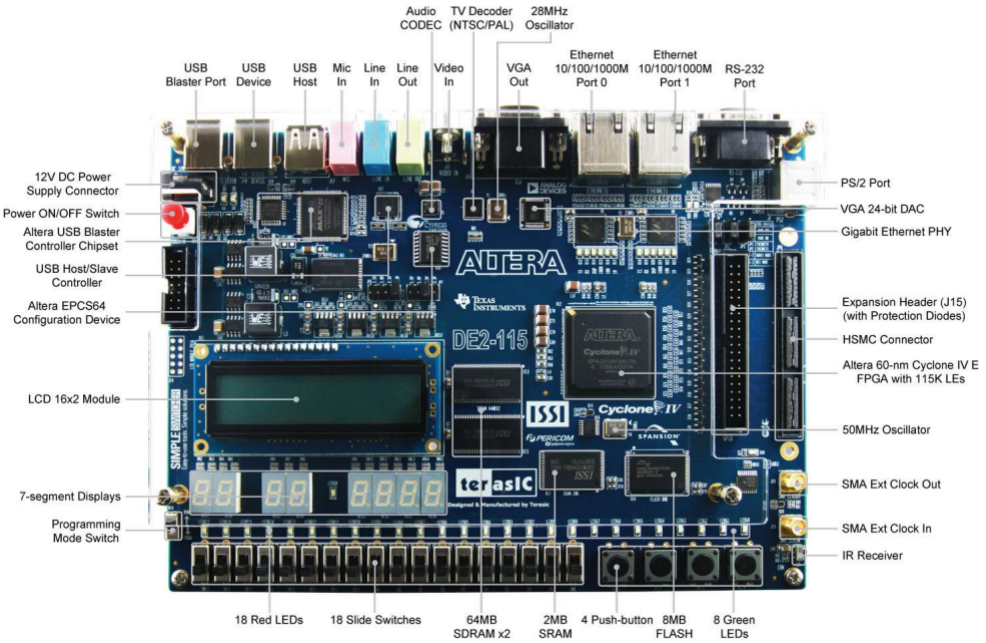
A linguagem Verilog é uma das HDL mais utilizadas no desenvolvimento de sistemas digitais complexos. Assim como o VHDL, o Verilog é capaz de descrever circuitos voltados para dispositivos como os FPGAs. Ele permite que o projetista documente e simule o comportamento de um sistema digital de forma eficiente, utilizando construções mais próximas à linguagens de programação, como a linguagem C. Com isso, torna-se possível representar equações booleanas, tabelas verdade e formas de onda, por meio da definição de módulos, estados, constantes e outras estruturas organizacionais do projeto (2).

3.2 FPGA

O FPGA (*Field Programmable Gate Array*) é uma tecnologia de circuito integrado que permite ao usuário programar e reconfigurar sua lógica interna mesmo após a fabricação. Em outras palavras, trata-se de um chip versátil que pode ser adaptado para realizar diferentes funções digitais, conforme a necessidade de cada projeto. Essa flexibilidade torna o FPGA útil em áreas como prototipagem de *hardware*, automação, telecomunicações e sistemas embarcados. A tecnologia surgiu comercialmente em 1983, com a empresa Xilinx, e representa uma alternativa eficiente e personalizável ao uso de circuitos fixos tradicionais (3).

O modelo específico para a implementação do projeto será o "Kit FPGA Altera DE2-115", apresentado na Figura 1.

Figura 1 – Kit FPGA Altera DE2-115



Fonte: DE2-115: User Manual (4)

3.3 Arquitetura de processadores

A arquitetura de processadores refere-se ao conjunto de atributos visíveis ao programador que definem o comportamento funcional de um processador. Em outras palavras, é o modelo e a organização conceitual de um sistema computacional, especificando como as instruções são interpretadas, como os dados são manipulados, e como os recursos internos do processador interagem (1).

De forma mais específica, as arquiteturas podem ser classificadas segundo o tipo de conjunto de instruções utilizado, como nas arquiteturas RISC ou CISC, e também pela organização da memória, sendo as mais comuns as arquiteturas de Harvard e de Von Neumann.

3.3.1 Arquiteturas CISC e RISC

A arquitetura RISC (*Reduced Instruction Set Computer*) é caracterizada por um conjunto reduzido e simplificado de instruções, com formatos padronizados e modos de endereçamento simples. Os sistemas RISC geralmente executam uma instrução por ciclo e priorizam operações entre registradores, o que favorece a eficiência do pipeline e o desempenho geral da execução.

Já a arquitetura CISC (*Complex Instruction Set Computer*) segue uma filosofia

oposta, oferecendo um conjunto de instruções mais amplo e complexo, com instruções que podem realizar tarefas mais elaboradas em um único comando. Essa abordagem visa reduzir a quantidade de instruções por programa, embora cada instrução possa demandar múltiplos ciclos para ser executada (1).

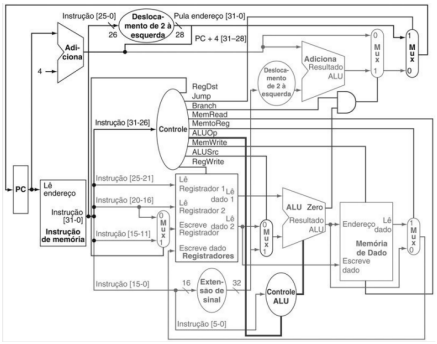
3.3.2 Arquiteturas de Von Neumann e Harvard

A arquitetura de Von Neumann caracteriza-se pelo uso de uma única memória e de um único barramento para armazenar e transferir tanto dados quanto instruções. Isso pode causar um gargalo de desempenho, já que as instruções e os dados competem pelo mesmo canal de acesso à memória. Em contraste, a arquitetura Harvard separa fisicamente a memória e os barramentos de dados e instruções, permitindo que ambos sejam acessados simultaneamente. Essa separação melhora a eficiência do sistema, especialmente em aplicações que exigem maior paralelismo e desempenho. (5)

3.3.3 Arquitetura MIPS

O MIPS (*Microprocessor without Interlocked Pipeline Stages*) é uma arquitetura de processador baseada no modelo RISC , idealizada por David Patterson em 1980 e projetada por John Hennessy em 1981. Seu desenvolvimento teve como objetivo criar uma arquitetura simples, com instruções de tamanho fixo e fácil decodificação, capaz de executar uma instrução por ciclo de *clock*. As instruções no MIPS possuem 32 bits e são organizadas em três formatos principais: R (registrador), I (imediato) e J (desvio) (6). A Figura 2 apresenta o caminho de dados mais básico projetado na bibliografia de Patterson e Hennessy (2017)(6).

Figura 2 – Caminho de dados básico para uma arquitetura MIPS



Fonte: Organização e projeto de computadores (6)

3.4 Instrução de máquina

Uma instrução de máquina é uma unidade básica de operação reconhecida diretamente pelo processador. Ela fornece ao *hardware* todas as informações necessárias para executar uma tarefa específica, como somar valores, mover dados ou tomar decisões de desvio no fluxo de execução. Cada instrução possui um "opcode", que identifica a operação a ser realizada, e pode conter referências a dados, conhecidos como operandos.

Os operandos podem ser classificados em operando fonte e operando destino. O operando fonte é o dado de entrada utilizado na operação — por exemplo, os números que serão somados. Já o operando destino é o local onde o resultado da operação será armazenado, como um registrador ou posição de memória. Esses operandos podem estar localizados em diferentes lugares: na memória principal, em registradores do processador, como valores imediatos dentro da própria instrução, ou em dispositivos de entrada e saída. (1)

3.5 Conjunto de instruções

Segundo Stallings (2010)(1), o conjunto de instruções é "a coleção de diferentes instruções que o processador por executar". Dentro desse conjunto, as instruções são organizadas em diferentes formatos, os quais definem a estrutura e os campos utilizados para representar cada operação.

3.5.1 Formato de instruções

O formato de instruções refere-se à maneira como essas instruções são codificadas em bits dentro do conjunto de instruções. Esse formato organiza os diversos campos de uma instrução — como o "opcode", registradores envolvidos, valores imediatos e endereços — dentro de um tamanho fixo ou variável, dependendo da arquitetura.

Na arquitetura MIPS, que adota o modelo RISC, as instruções possuem um tamanho fixo de 32 bits e se dividem em três formatos principais. O formato R é utilizado em operações realizadas entre registradores, como somas ou subtrações. Nesse formato, especificam-se dois operandos de origem (denominados "rs"e "rt") e um registrador de destino (rd), além do campo "opcode", que identifica a categoria da instrução, "funct", que define a operação exata, e "shamt", que indica o deslocamento em operações de deslocamento lógico.

Já o formato I é voltado para instruções que utilizam valores imediatos ou acessam a memória. Ele inclui os registradores "rs"e "rt", juntamente com um valor imediato de 16 bits, permitindo, por exemplo, carregar ou armazenar dados em posições de memória.

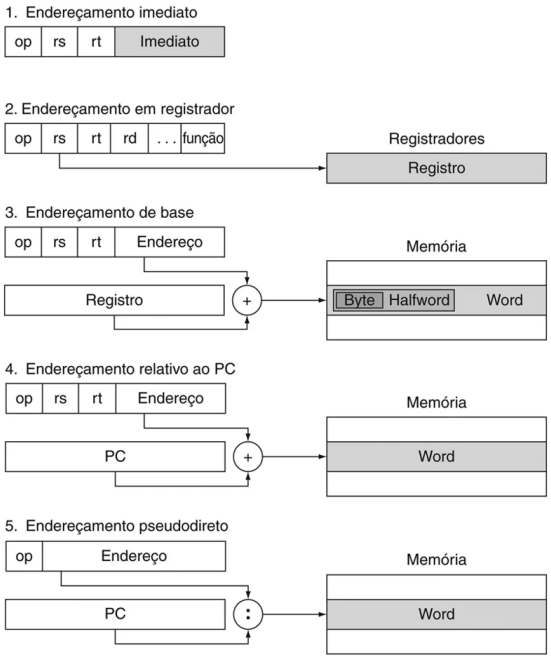
Por fim, o formato J é destinado a instruções de desvios incondicionais. Seu campo principal, além do opcode, é um endereço de 26 bits, que representa o destino para o qual o fluxo de execução deve saltar. (7)

3.6 Modos de Endereçamento

Os modos de endereçamento correspondem aos diferentes mecanismos utilizados por uma arquitetura de processador para localizar os operandos necessários à execução das instruções (6). A escolha dos modos suportados influencia diretamente a flexibilidade, a complexidade e a eficiência do conjunto de instruções. Cada modo define uma forma específica de interpretar os bits da instrução para acessar dados, seja nos registradores ou na memória principal, apresentando vantagens e desvantagens distintas quanto ao desempenho e à capacidade de endereçamento.

Na arquitetura MIPS, segundo Patterson e Hennesy (2017)(6), o modos de endereçamento são cinco: Endereçamento imediato, Endereçamento por registrador, Endereçamento de base, Endereçamento relativo ao PC e Endereçamento pseudodireto. Todas essas categorias são esquematizadas na Figura 3.

Figura 3 – Modos de endereçamento



Fonte: Organização e projeto de computadores (6)

3.6.1 Endereçamento imediato

O endereçamento imediato é o modo mais simples, no qual o operando está contido diretamente na instrução. Esse tipo de endereçamento é amplamente utilizado para carregar

valores constantes ou inicializar variáveis. Sua principal vantagem é a ausência de acessos à memória para buscar o dado, o que proporciona maior velocidade de execução. Contudo, o tamanho do operando é limitado pela quantidade de bits disponíveis na própria instrução, o que impõe restrições à sua aplicabilidade em operações com dados de maior dimensão (6).

3.6.2 Endereçamento em registrador

No endereçamento por registrador, os operandos estão localizados em registradores internos do processador. Esse modo é bastante eficiente, pois o acesso a registradores é significativamente mais rápido do que à memória principal. Por esse motivo, é comum em instruções do tipo R e I. Em contrapartida, a principal limitação desse modo reside na quantidade finita de registradores disponíveis, o que restringe o espaço de endereçamento acessível diretamente (1).

3.6.3 Endereçamento de base ou deslocamento

O endereçamento por base com deslocamento, também conhecido como endereçamento com offset, calcula o endereço efetivo do operando somando-se uma constante (deslocamento) ao valor de um registrador base. Esse registrador normalmente contém o endereço de referência para estruturas como vetores ou campos de registros. Esse modo permite maior flexibilidade na manipulação de dados armazenados em posições de memória consecutivas e é amplamente utilizado em instruções de carga e armazenamento, como load e store. Sua desvantagem reside na maior complexidade envolvida no cálculo do endereço efetivo (6).

3.6.4 Endereçamento relativo ao PC

O endereçamento relativo ao contador de programa (PC) segue o mesmo princípio do modo base com deslocamento, porém utiliza o valor do PC (Program Counter) como base. O endereço de destino é obtido somando-se ao PC um valor constante presente na instrução. Esse modo é essencial para instruções de desvio condicional e incondicional dentro de trechos de código relativamente curtos, sendo comum em instruções do tipo I. Sua principal vantagem é possibilitar saltos posicionais sem exigir o armazenamento de endereços absolutos, o que favorece a relocação do código em memória (6).

3.6.5 Endereçamento pseudodireto

O endereçamento pseudodireto é utilizado principalmente em instruções do tipo J (jump), como j e jal. Nesse modo, o endereço de salto é formado a partir da concatenação de 26 bits presentes na instrução com os 4 bits mais significativos do PC incrementado em

4, sendo os dois bits menos significativos fixados em zero para garantir o alinhamento de palavras. Essa técnica permite representar um espaço de endereçamento de 28 bits dentro de uma instrução de 32 bits. Vale ressaltar que instruções como jr (jump register), embora realizem saltos, utilizam registradores para obter o endereço de destino e pertencem, portanto, ao formato R (6).

3.7 Circuitos Combinacionais

Circuitos combinacionais são aqueles cujo valor de saída depende exclusivamente dos valores atuais das entradas, sem considerar estados anteriores. Eles realizam operações lógicas e aritméticas básicas, como somadores, multiplexadores, decodificadores e codificadores. A característica essencial desse tipo de circuito é a ausência de memória interna; sua resposta é instantânea e determinada pela função lógica que implementam (8).

Na prática, um circuito combinacional pode ser descrito por uma função booleana que relaciona diretamente as variáveis de entrada às saídas. A construção desses circuitos envolve a utilização de portas lógicas (AND, OR, NOT, etc.) para implementar expressões lógicas específicas, possibilitando o processamento imediato dos dados conforme a lógica projetada.

3.8 Circuitos Sequenciais

Diferentemente dos circuitos combinacionais, os circuitos sequenciais possuem memória e sua saída depende não apenas das entradas atuais, mas também do histórico de entradas e do estado anterior do circuito. Essa característica permite que sejam utilizados para construir elementos que armazenam informação, como flip-flops, registradores e contadores (8).

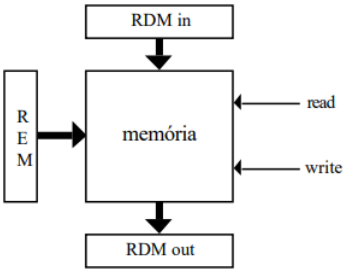
Segundo Tocci (2016)(8), os circuitos sequenciais são divididos em síncronos e assíncronos. A maioria dos sistemas digitais modernos utiliza circuitos sequenciais síncronos, que dependem de um sinal de *clock* para sincronizar as mudanças de estado. Esse sinal garante que as alterações de estados ocorram de forma ordenada, permitindo o controle preciso do fluxo de dados e execução das operações.

Esses circuitos são essenciais para a construção de unidades de controle, processadores, memórias e outros componentes complexos de sistemas digitais, pois possibilitam a implementação de comportamento dinâmico e sequencial nos sistemas.

3.9 Memória

A memória em um sistema computacional é composta por unidades responsáveis por guardar informações temporariamente. Essas informações são organizadas em blocos chamados de palavras (ou *words*, do inglês), sendo que cada uma ocupa uma localização específica dentro da memória. Essa localização é identificada por um endereço único, permitindo que o processador acesse diretamente o dado desejado. As palavras armazenadas podem conter tanto instruções que serão executadas quanto dados a serem manipulados. Em termos de arquitetura, o que se considera relevante é o número de palavras disponíveis e o tamanho de cada palavra, medido em bits (2).

Figura 4 – Modelo esquemático da memória



Fonte: Fundamentos de Arquitetura de Computadores (2)

A estrutura da memória pode ser representada conforme ilustrado na Figura 4. Nela, o componente "REM"(Registrador de Endereço de Memória) armazena o endereço que será utilizado para acessar uma posição específica da memória. O campo "RDM in"(Registrador de Dados de Memória) representa o dado que será escrito na memória, enquanto "RDM out"indica o valor lido a partir do endereço indicado. Além disso, note que existem dois sinais de controle: *read* para leitura e *write* para escrita.

Também, as memórias podem ser classificadas como do tipo RAM (*Random Acess Memory*) ou do tipo ROM (*Read Only Memory*).

3.9.1 Memórias RAM

A memória do tipo RAM é uma forma de armazenamento volátil que permite acesso aleatório a qualquer posição de memória, com tempos de leitura e escrita semelhantes para todas as localizações. Essa característica torna a RAM adequada para armazenar dados e instruções temporárias durante a execução de programas, sendo essencial para o funcionamento dinâmico do sistema (1).

3.9.2 Memórias ROM

A memória ROM é um tipo de memória não volátil, cujo conteúdo é definido no momento da fabricação ou por meio de processos específicos de gravação. Como o próprio nome indica, essa memória é destinada principalmente à leitura, sendo utilizada para armazenar informações permanentes ou semipermanentes.

3.10 Registradores

Os registradores são componentes fundamentais no funcionamento interno de um processador. Eles consistem em elementos digitais capazes de armazenar dados temporariamente, sendo controlados por sinais que determinam o momento em que novos valores devem ser carregados. Esses dispositivos operam de forma extremamente rápida e são utilizados em diversas operações internas da CPU (2).

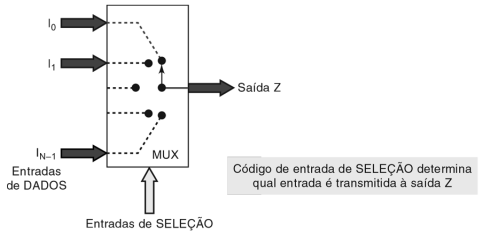
3.10.1 *Program Counter (PC)*

Dentre os diversos tipos de registradores existentes, destaca-se o apontador de instruções, também conhecido como contador de programa (PC – Program Counter). Sua função principal é manter o endereço da próxima instrução a ser executada. Em arquiteturas básicas, as instruções são armazenadas em posições de memória consecutivas, o que permite ao PC simplesmente incrementar seu valor para apontar para a instrução seguinte. A largura desse registrador, ou seja, a quantidade de bits que ele comporta, está diretamente relacionada ao tamanho do espaço de endereçamento de memória da arquitetura utilizada (2).

3.11 Multiplexadores (MUX)

O multiplexador, também conhecido como seletor de dados, é um circuito lógico utilizado para direcionar um entre vários sinais de entrada digitais para uma única saída. Ele funciona como uma espécie de chave controlada por sinais chamados de entradas de seleção, que determinam qual dos dados de entrada será transmitido. Em seu funcionamento, o multiplexador avalia o valor das entradas de controle e, com base nisso, escolhe o dado correspondente para ser encaminhado à saída (8). Um multiplexador está representado na Figura 5.

Figura 5 – Modelo esquemático de um multiplexador



Fonte: Sistemas Digitais (8)

3.12 Unidade de Controle (UC)

O controle das operações internas em um processador exige uma coordenação precisa do momento e do caminho pelo qual os dados transitam entre os diversos componentes. Para isso, existe a Unidade de Controle, responsável por gerar os sinais que comandam essas movimentações. Cada um desses sinais ativa uma ação específica, chamada de microoperação. Essas microoperações podem envolver, por exemplo, o carregamento de dados em um registrador, a escolha de um valor para entrada em um bloco funcional, a solicitação de leitura ou escrita na memória, a definição da operação a ser executada pela ULA ou mesmo a ativação de determinadas portas lógicas (2).

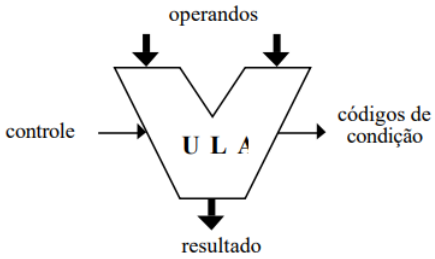
3.13 Unidade Lógica Aritmética (ALU)

A Unidade Lógica e Aritmética (ULA ou ALU), representada na Figura 6, é o componente do processador encarregado de realizar operações matemáticas simples, como somas, subtrações, e também operações lógicas, como AND e OR. Além disso, pode executar deslocamentos e rotações de bits. Essas operações são consideradas básicas, mas podem ser combinadas em sequência, permitindo a execução de instruções mais complexas, como multiplicações e divisões.

O funcionamento da ULA depende de sinais de controle, que indicam qual operação deve ser realizada naquele ciclo de execução. A ULA processa os operandos recebidos e devolve o resultado, além de gerar sinais adicionais chamados de "códigos de condição". Esses códigos indicam características do resultado, como se ele é zero, negativo ou se houve um estouro (*overflow*), por exemplo.

Vale destacar que a ULA não armazena os operandos nem o resultado final; ela apenas realiza o cálculo temporariamente. Por fim, uma ULA deve ser caracterizada pelo tamanho dos dados que consegue processar (por exemplo, 32 ou 64 bits), a variedade de operações suportadas e os tipos de sinais de condição que pode emitir (2).

Figura 6 – Modelo esquemático da ULA



Fonte: Fundamentos de Arquitetura de Computadores (2)

4 Desenvolvimento

4.1 Características do processador

O processador desenvolvido neste projeto segue a arquitetura RISC (*Reduced Instruction Set Computer*), sendo baseado no modelo MIPS 32 monociclo, conforme proposto por Patterson e Hennessy (2005) (7). A implementação utiliza um conjunto de instruções fixo de 32 bits e conta com 32 registradores de propósito geral, cada um com largura de 32 bits.

A organização da memória adota a arquitetura Harvard, com separação física entre a memória de instruções e a memória de dados. Essa estrutura é essencial para garantir que as instruções e os dados possam ser acessados simultaneamente durante um único ciclo de operação do processador.

Embora o modelo arquitetural do processador seja monociclo — ou seja, todas as instruções são executadas em um único ciclo de *clock* —, a implementação física no FPGA, em função das características das memórias internas disponíveis, exige a utilização de dois sinais de *clock* distintos: um para leitura (50 MHz) e outro para escrita (que opera seguindo o mesmo ciclo de *clock* do caminho de dados). Essa divisão se faz necessária porque tais memórias operam de forma síncrona, exigindo controle separado para as operações de leitura e escrita. Dessa forma, a leitura combinacional — ideal para a abordagem monociclo — não seria compatível com o uso direto das memórias físicas do FPGA. Caso fosse empregado um único sinal de *clock*, o processador apresentaria atrasos na execução das instruções, comprometendo seu funcionamento correto.

Acerca da memória de instruções, ela será do tipo ROM e organizada com 32 colunas, correspondentes aos 32 bits que compõem cada instrução, e 256 linhas, permitindo o armazenamento de até 256 instruções distintas. Essa estrutura é do tipo *word-addressable*, o que implica numa diferença em relação à arquitetura descrita por Patterson e Hennessy (2005)(7), na qual a memória é *byte-addressable* e cada posição armazena um byte, exigindo um incremento de PC de 4 em 4 para acessar instruções de 32 bits. Na arquitetura aqui desenvolvida, como cada posição representa diretamente uma palavra de 32 bits, o incremento do contador de programa será realizado com PC + 1, tornando o acesso à próxima instrução de maneira mais direta.

A arquitetura geral é composta por um contador de programa (PC), responsável por armazenar o endereço da próxima instrução a ser executada, por uma memória de instruções que contém o programa, por um banco de registradores com 32 posições de 32 bits, que permite o acesso para leitura e escrita, e por uma unidade lógica e aritmética

(ALU), responsável pela execução das operações aritméticas e lógicas previstas no conjunto de instruções. Além disso, há uma memória de dados do tipo RAM separada, destinada ao armazenamento e acesso das informações manipuladas durante a execução das instruções, conforme a arquitetura Harvard adotada.

Além desses elementos centrais, a arquitetura conta com uma Unidade de Controle (UC) principal, que interpreta o campo de opcode das instruções e gera os sinais de controle apropriados para o fluxo de dados no processador. Também, multiplexadores são empregados em diversos pontos do caminho de dados para permitir a seleção entre diferentes fontes de informação. Uma Unidade de Controle específica da ALU também é utilizada, sendo responsável por definir qual operação a ALU deverá realizar com base na instrução recebida. Fora isso, um Extensor de Sinal é utilizado para a formatar os valores binários de imediatos tanto de instruções quanto de *input* de 16 para 32 bits. Por fim, uma Unidade Lógica para *Branch* é inserida, sendo ela dedicada para determinar os desvios condicionais, e somadores são implementados para realizar incrementos no PC ou cálculos de endereços de salto.

Conforme o objetivo geral do projeto, a implementação do processador será realizada em um kit FPGA. Para permitir a comunicação com o ambiente externo, serão utilizadas instruções específicas de entrada (*input*) e saída (*output*). A instrução de entrada possibilitará carregar um valor diretamente em um registrador, a partir dos *switches* disponíveis na placa FPGA, os quais, em conjunto, representam um número binário. Por sua vez, a instrução de saída permitirá exibir o conteúdo de um registrador nos *displays* de 7 segmentos da placa, viabilizando a visualização direta dos dados armazenados no registrador especificado.

Em relação ao formato das instruções, será adotado o mesmo conjunto proposto por Patterson e Hennessy (2005)(7), o qual define três formatos principais: tipo R, tipo I e tipo J. Além disso, os modos de endereçamento implementados seguirão os mesmos descritos pelos autores, sendo eles, então, o endereçamento imediato, por registrador, com base e deslocamento, relativo ao contador de programa (PC) e pseudodireto. No entanto, serão incluídas novas instruções ao conjunto original, o que implicará em adaptações no *datapath* e possivelmente na lógica de controle, a fim de suportar essas extensões funcionais.

4.2 Conjunto de Instruções

A arquitetura desenvolvida adota os três formatos clássicos de instrução da arquitetura MIPS: formato R, formato I e formato J, apresentados nas Tabelas 1, 2 e 3, respectivamente.

Tabela 1 – Formato de instruções do tipo R

Tamanho (bits)	6	5	5	5	5	6
Campo	opcode	rs	rt	rd	shamt	funct
Bits	31 - 26	25 - 21	20 - 16	15 - 11	10 - 6	5 - 0

Fonte: O Autor

Tabela 2 – Formato de instruções do tipo I

Tamanho (bits)	6	5	5	16
Campo	opcode	rs	rt	imediato
Bits	31 - 26	25 - 21	20 - 16	15 - 0

Fonte: O Autor

Tabela 3 – Formato de instruções do tipo J

Tamanho (bits)	6	26
Campo	opcode	target
Bits	31 - 26	25 - 0

Fonte: O Autor

Cada instrução é projetada de acordo com a estrutura que melhor atende à sua finalidade. Assim, o conjunto de instruções do processador foi definido conforme essas tabelas, respeitando o alinhamento entre a estrutura da instrução e o tipo de operação realizada. Todas as instruções do conjunto de instruções são apresentados na Tabela 4.

Vale ressaltar que a instrução de *input* será capaz de comportar um número binário de até 16 bits, uma vez que, para essa finalidade, serão usados 16 *switches* dos 18 presentes no kit FPGA. No caso da instrução de *output*, o kit conta com 8 *displays* de 7 segmentos, sendo 7 deles utilizados para representação numérica. Assim, para saída, a maior representação binária possivel será $(100110001001011001111111)_2$, o que representa um número de 24 bits.

Outras instruções que merecem destaque são as de multiplicação (MUL) e divisão (DIV). Diferentemente das instruções tradicionais em arquiteturas MIPS, como a proposta por Patterson e Hennessy (2005) (7), essas instruções representarão diretamente valores de até 32 bits, sem realizar a separação entre a parte mais significativa e a menos significativa em registradores distintos. Ou seja, não haverá o uso de registradores dedicados para armazenar os 64 bits resultantes da operação.

Com relação à distinção das instruções, para cada uma delas, é necessário determinar os códigos que as identificam. No caso das instruções de formato R, devem ser especificados tanto o campo "opcode"quanto o campo "funct". O opcode é utilizado para indicar que se trata de uma instrução do tipo R, enquanto o funct especifica qual operação será executada, diferenciando, por exemplo, entre instruções como add, mul ou sll.

Tabela 4 – Conjunto de instruções

Instrução	**Operação**	**Explicação**	**Tipo**
Aritméticas:			
ADD	rd = rs + rt	Soma	R
ADDI	rt = rs + imediato	Soma com imediato	I
SUB	rd = rs - rt	Subtração	R
MUL	rd = rs * rt	Multiplicação	R
DIV	rd = rs / rt	Divisão	R
Lógicas:			
AND	rd = rs & rt	"E"lógico	R
ANDI	rt = rs & imediato	"E"com imediato	I
OR	rd = rs | rt	"OU"lógico	R
ORI	rt = rs | imediato	"OU"com imediato	I
XOR	rd = rs $\oplus$ rt	"XOR"lógico	R
XORI	rt = rs $\oplus$ imediato	"XOR"com imediato	I
NOR	rd = $\sim$(rs | rt)	"NOR"lógico	R
Deslocamento:			
SLL	rd = rt <<shamt	Desloca para esquerda	R
SLR	rd = rt >>shamt	Desloca para a direita	R
Transferência de Dados:			
LW	rt = Mem[rs + offset]	Carrega uma palavra	I
SW	Mem[rs + offset] = rt	Armazena uma palavra	I
LUI	rt = imediato <<16	Carrega um imediato nos 16 bits superiores e zera os inferiores.	I
Desvio Condicional:			
BEQ	se rs == rt, PC = PC+ offset	Desvia se igual	I
BNE	se rs != rt, PC = PC + offset	Desvia se diferente	I
BLT	se rs <rt, PC = PC + offset	Desvia se menor	I
BGT	se rs >rt, PC = PC + offset	Desvia se maior	I
BLE	se rs <= rt, PC = PC + offset	Desvia se menor ou igual	I
BGE	se rs >= rt, PC = PC + offset	Desvia se maior	I
Desvio Incondicional:			
J	PC = endereço	Pula de endereço	J
JAL	reg = PC; PC = endereço	Chamada de sub-rotina	J
JR	PC = Reg[rs]	Retorno de sub-rotina	R
Controle:			
NOP	-	Sem operação	J
HALT	-	Parada	J
Entrada e Saída:			
IN	rt = imediato (*switches*)	Carrega um imediato dado pelo input	I
OUT	saída = Reg[rs]	Envia conteúdo de um registrador para a saída	I

Fonte: O Autor

Tabela 5 – Instruções de formato R com seus valores de opcode e funct

Opcode	**Funct**	**Instrução**
000000	000000	ADD
000000	000001	SUB
000000	000010	MUL
000000	000011	DIV
000000	000100	AND
000000	000101	OR
000000	000110	XOR
000000	000111	NOR
000000	001000	JR
000000	001001	SLL
000000	001010	SRL

Fonte: O Autor

Para as instruções de formato I e J, apenas o campo "opcode"é necessário para identificar a operação, visto que esse campo já é suficiente para diferenciar todas as instruções desses formatos. Assim, os códigos correspondentes a cada uma dessas instruções foram definidos, conforme apresentado nas Tabelas 5, 6 e 7.

Tabela 6 – Instruções de formato I com seus valores de opcode e funct

Opcode	**Instrução**
000001	ADDI
000010	ANDI
000011	ORI
000100	XORI
000101	LW
000110	SW
000111	LUI
001000	BEQ
001001	BNE
001010	BLT
001011	BGT
001100	BLE
001101	BGE
001110	IN
001111	OUT

Fonte: O Autor

Tabela 7 – Instruções de formato J com seus valores de opcode e funct

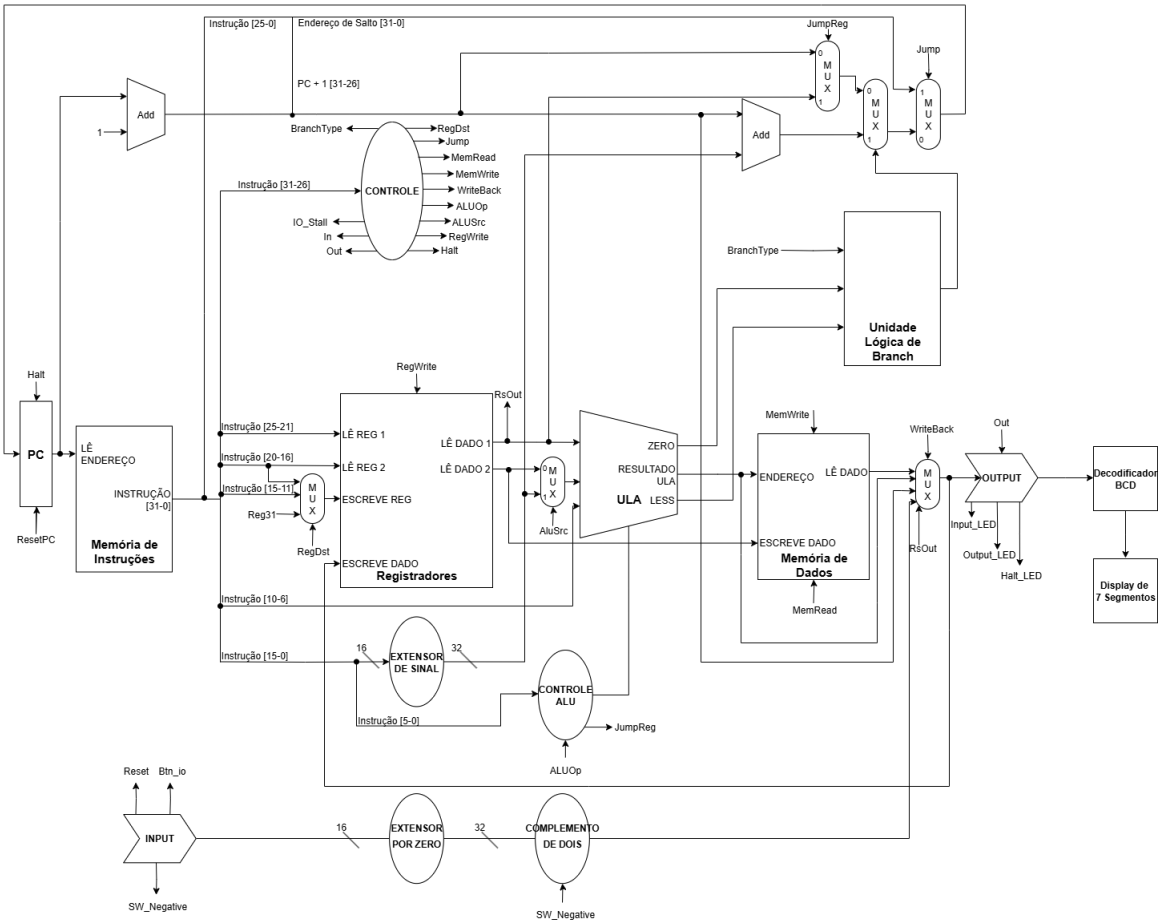
Opcode	**Instrução**
010000	J
010001	JAL
010010	NOP
010011	HALT

Fonte: O Autor

4.3 *Datapath*

O *datapath* (caminho de dados) representa o percurso dos dados pelos componentes do processador necessários à execução de cada instrução. Considerando as características da arquitetura definidas anteriormente, bem como a ampliação do conjunto de instruções, foi projetado o *datapath* apresentado na Figura 7. Vale-se ressaltar, novamente, que a base do projeto segue o modelo proposto por Patterson e Hennessy (2005, p. 231)(7), com adaptações visando o suporte a um conjunto de instruções mais abrangente.

Figura 7 – *Datapath* do processador



Fonte: O Autor

Cada formato e cada instrução exige diferentes componentes e caminhos para seguir. Para conseguir atender a particularidade de cada instrução, diferentes multiplexadores são posicionados ao longo do caminho de dados. Assim, para que as seleções ocorram corretamente ao longo do caminho de dados, são necessários sinais de controle como entrada. A responsabilidade por definir esses sinais cabe à Unidade de Controle, que os determina a partir do campo de opcode da instrução em execução. Com isso, a UC gera os sinais adequados para conduzir a execução da instrução. Todos os sinais gerados pela Unidade de Controle estão descritos na Tabela 8.

Tabela 8 – Descrição dos sinais de controle

Sinais de Controle (Unidade de Controle)	**000**	**001**	**010**	**011**	**100**	**101**	**111**
RegDst	Sem efeito	Define o registrador de destino como o campo rt (bits {[}20-16{]})	Define o registrador de destino como o campo rd (bits {[}15-11{]})	Define o registrador de destino como o registrador 31	-	-	-
Jump	Atualiza o Program Counter com o valor obtido pelos MUXs anteriores	Salto incondicional com valor concatenado de {[}25-0{]} e PC+1	-	-	-	-	-
MemRead	Sem efeito	Ativa a leitura da memória no endereço 'ENDEREÇO'	-	-	-	-	-
MemWrite	Sem efeito	Ativa a escrita do dado 'ESCREVE DADO' no endereço 'ENDEREÇO'	-	-	-	-	-
ALUOp	Operação de soma na ALU	Operação lógica 'AND' na ALU	Operação lógica 'OR' na ALU	Operação lógica 'XOR' na ALU	Operação lógica 'Subtração' na ALU	Deslocamento à esquerda de 16 bits no imediato de entrada da ALU	Instrução do tipo R identificada para a UC da ALU
ALUSrc	Seleciona dado do banco de registradores para a ALU	Imediato da instrução estendido para 32 bits	-	-	-	-	-
WriteBack	Dado lido na memória	Resultado da ALU	PC + 1	Dado do módulo de input estendido	Dado do registrador "rs"	-	-
RegWrite	Sem efeito	Escrita no registrador de endereço 'ESCREVE DADO'	-	-	-	-	-
Halt	Sem efeito	Interrupção do Program Counter	-	-	-	-	-
BranchType	Sem efeito	Determina o sinal de desvio BEQ	Determina o sinal de desvio BNE	Determina o sinal de desvio BGT	Determina o sinal de desvio BLT	Determina o sinal de desvio BLE	Determina o sinal de desvio BGE
IN	Sem efeito	Instrução de input ativa	-	-	-	-	-
OUT	Sem efeito	Instrução de output ativa	-	-	-	-	-
IO_STALL	Sem efeito	Interrupção do PC para instrução de entrada e saída	-	-	-	-	-

Fonte: O Autor

O percurso inicial de uma instrução no datapath é comum a todas as instruções: a memória de instruções realiza a leitura do endereço contido no PC, acessando, assim, a próxima instrução a ser executada. Após o acesso à instrução, seu campo "opcode"é enviado à Unidade de Controle, que definirá os sinais de controle correspondentes. A partir desse ponto, o fluxo de execução passa a variar conforme as particularidades de cada instrução.

O mapeamento dos valores de controle para cada instrução estão descritos na Tabela 9.

Tabela 9 – Sinais de controle por instrução

Entrada/Saída	**Nome do sinal**	**formato R**	**lw**	**lui**	**sw**	**beq**	**bne**	**blt**	**bgt**	**ble**	**bge**	**addi**	**andi**	**ori**	**xori**	**in**	**out**	**j**	**jal**	**nop**	**halt**
Entradas	Op5	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
	Op4	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	1	1	1
	Op3	0	0	0	0	1	1	1	1	1	1	0	0	0	0	1	1	0	0	0	0
	Op2	0	1	1	1	0	0	0	0	1	1	0	0	0	1	1	1	0	0	0	0
	Op1	0	0	1	0	0	0	1	1	0	0	0	1	1	0	1	1	0	0	1	1
	Op0	0	1	1	1	0	1	0	1	0	1	1	0	1	0	0	1	0	1	0	1
Saídas	RegDst1	1	0	0	x	x	x	x	x	x	x	0	0	0	0	0	x	x	1	0	x
	RegDst0	0	1	1	x	x	x	x	x	x	x	1	1	1	1	1	x	x	1	0	x
	AluSrc	0	1	1	1	0	0	0	0	0	0	1	1	1	1	x	x	x	x	0	0
	MemRead	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
	MemWrite	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
	WriteBack2	0	0	0	x	x	x	x	x	x	x	0	0	0	0	0	1	x	0	0	x
	WriteBack1	0	0	0	x	x	x	x	x	x	x	0	0	0	0	1	0	x	1	0	x
	WriteBack0	1	0	1	x	x	x	x	x	x	x	1	1	1	1	1	0	x	0	0	x
	RegWrite	1	1	1	0	0	0	0	0	0	0	1	1	1	1	1	0	0	1	0	0
	BranchType2	0	0	0	0	0	0	0	1	1	1	0	0	0	0	0	0	0	0	0	0
	BranchType1	0	0	0	0	0	1	1	0	0	1	0	0	0	0	0	0	0	0	0	0
	BranchType0	0	0	0	0	1	0	1	0	1	0	0	0	0	0	0	0	0	0	0	0
	Jump	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	1	0	0
	ALUOp2	1	0	1	0	1	1	1	1	1	1	0	0	0	0	x	x	x	x	x	x
	ALUOp1	1	0	0	0	0	0	0	0	0	0	0	0	1	1	x	x	x	x	x	x
	ALUOp0	1	0	1	0	0	0	0	0	0	0	0	1	0	1	x	x	x	x	x	x
	Out	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0
	In	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0
	IO_Stall	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	1	0	0	0	0
	Halt	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1

Fonte: O Autor

Além dos sinais de controle gerais, as instruções que fazem uso da ALU exigem

que ela execute operações específicas conforme o tipo de instrução. Para isso, a ALU é acompanhada por uma Unidade de Controle dedicada, responsável por interpretar o campo "ALUOp", emitido pela Unidade de Controle principal. Este campo possui valores distintos que determinam qual operação lógica ou aritmética a ALU deverá realizar.

No caso das instruções de formato R, todas compartilham o mesmo opcode, o que implica em um mesmo valor de ALUOp atribuído pela Unidade de Controle. Desse modo, convencionou-se utilizar o valor “111” para esse tipo de instrução. A distinção entre as operações, portanto, será feita com base no campo "funct", analisado exclusivamente pela Unidade de Controle da ALU, que determina a operação correspondente.

Adicionalmente, o sinal "JumpReg"(utilizado em "JR"), por ser necessário para o funcionamento de uma única instrução de formato R, também será definido nessa unidade, o que será mais detalhado adiante.

A Tabela 10 apresenta o mapeamento entre os valores de "ALUOp"e "funct"e as operações resultantes na ALU.

Tabela 10 – Sinais de operação da ALU

ALUOp			**Funct**						**Operação**	**JumpReg**
ALUOp2	**ALUOp1**	**ALUOp0**	**F5**	**F4**	**F3**	**F2**	**F1**	**F0**		
0	0	0	x	x	x	x	x	x	0000 (Soma)	0
0	0	1	x	x	x	x	x	x	0100 (AND)	0
0	1	0	x	x	x	x	x	x	0101 (OR)	0
0	1	1	x	x	x	x	x	x	0110 (XOR)	0
1	0	0	x	x	x	x	x	x	0001 (Subtração)	0
1	0	1	x	x	x	x	x	x	1010 (<<16)	0
1	1	1	0	0	0	0	0	0	0000 (Soma)	0
1	1	1	0	0	0	0	0	1	0001 (Subtração)	0
1	1	1	0	0	0	0	1	0	0010 (Multiplicação)	0
1	1	1	0	0	0	0	1	1	0011 (Divisão)	0
1	1	1	0	0	0	1	0	0	0100 (AND)	0
1	1	1	0	0	0	1	0	1	0101 (OR)	0
1	1	1	0	0	0	1	1	0	0110 (XOR)	0
1	1	1	0	0	0	1	1	1	0111 (NOR)	0
1	1	1	0	0	1	0	0	0	XXXX	1
1	1	1	0	0	1	0	0	1	1000 (SLL)	0
1	1	1	0	0	1	0	1	0	1001 (SRL)	0

Fonte: O Autor

4.4 Implementação do Processador

Nesta etapa do projeto, a implementação foi realizada utilizando a linguagem SystemVerilog no ambiente *Quartus Prime Lite*, com o objetivo de viabilizar seu funcionamento no kit FPGA Altera Cyclone IV DE2-115. As implementações seguiram o caminho de dados projetado na Figura 7, respeitando as interligações definidas entre os blocos.

Os módulos desenvolvidos serão destrinchados nessa seção e estarão disponíveis

para consulta no Apêndice do relatório.

4.4.1 Processador

O processador implementado é composto pela integração de vários módulos individuais, que em conjunto permitem seu funcionamento. Essa junção dos elementos do processador foi feita no módulo `mips_processor`, apresentado no Apêndice B.

As entradas desse módulo incluem os sinais de *clock*, com `clk` sendo o responsável pela sincronização geral do processador e `read_clk` dedicado para as leituras nas memórias. Também, o sinal de `reset`, o botão de controle para operações de entrada/saída (`btn_io`), um vetor de 16 bits de `switches` utilizado para entrada de dados e o sinal `sw_negative`, que indica se o valor de entrada deve ser interpretado como negativo.

Internamente, o processador opera com diversos sinais de controle, responsáveis por coordenar a comunicação entre os módulos e definir o comportamento das instruções. Todos eles foram definidos como `logic`, junto de outros dados intermediários entre as integrações dos módulos que compõem o processador.

Além das integrações dos módulos, dentro do programa também ocorrem três atribuições combinacionais de dados. As duas primeiras, presentes nas linhas 24 e 25, promovem a padronização do formato do dado de entrada vindos dos *switches*, primeiro fazendo a extensão sem sinal de seu tamanho para o padrão de 32 bits para que, em seguida, seja definido se o valor de *input* é do formato negativo ou não. O outro `assign` é feito na linha 147, em que é responsável por atualizar a saída `flag_negative`, usada posteriormente para determinar a impressão do sinal de negativo no *display* do FPGA.

Acerca das saídas, elas são compostas pelo dado de saída do processador (`data_out`), que representa o resultado da execução de instruções e um conjunto de *flags*: `flag_in` e `flag_out`, que indicam operações de entrada e saída; `flag_negative`, que informa se o dado de saída é negativo; e `flag_halt`, que sinaliza a paralisação do processador. Todas essas *flags* serão utilizadas no módulo responsável por configurar o sistema como um todo, unindo a comunicação externa do kit FPGA com a lógica interna do processador.

A integração desses módulos pode ser visualizada no caminho de dados representado pela Figura 7, sendo que cada componente — `program_counter` (contador de programa), `instruction_ROM` (memória de instruções), `control_unit` (unidade de controle), `register_file` (banco de registradores), `mux_write_reg` (multiplexador de seleção de registrador de escrita), `mux_alusrc` (multiplexador da entrada da ALU), `mux_write_back` (multiplexador de seleção de dado de escrita e saída), `sign_extension_imm` (extensor de sinal imediato), `ALU_control` (unidade de controle da ALU), `ALU` (unidade lógica e aritmética), `branch_logic_unit` (unidade de controle de desvio), `data_RAM` (memória de dados) e `pc_update_logic` (lógica de atualização do PC) — será discutido individualmente nas

subseções subsequentes.

4.4.2 *Program Counter*

O módulo `program_counter`, apresentado no Apêndice C, tem como principal função armazenar e atualizar o valor do contador de programa (PC), responsável por indicar o endereço da próxima instrução a ser buscada na memória. Esse componente é essencial para garantir o fluxo sequencial da execução das instruções em um processador do tipo MIPS. O módulo opera de forma síncrona, sendo acionado pela borda de subida do sinal de *clock*. Ele recebe como entrada os sinais de controle `reset`, `halt` e `io_stall`, o novo valor do contador (`pc_in`), o clock do processador e `clk_manual` — usado em instruções de input e output. A saída do módulo é o valor atual do PC (`pc_out`), utilizado como endereço para acesso à memória de instruções.

Quando o sinal reset está ativado, o valor do contador é imediatamente reinicializado para zero, o que permite reiniciar a execução do programa a partir do início. Caso o reset não esteja ativo e o sinal halt esteja desativado, o módulo atualiza o valor do PC com o conteúdo de `pc_in`, garantindo a progressão normal do programa. Caso o sinal halt esteja ativado, o valor de `pc_out` permanece inalterado, impedindo a execução de novas instruções. Este comportamento possibilita o controle da execução em situações específicas, como paradas forçadas ou estados de espera.

Além do funcionamento descrito, o módulo incorpora uma lógica de seleção do sinal de clock utilizado na atualização do PC, motivada pela implementação das operações de entrada e saída do processador. Nessas situações, é necessário interromper temporariamente o avanço do processador, aguardando a inserção de dados de entrada ou a autorização do usuário para prosseguir após uma instrução de saída. Para isso, quando o sinal `io_stall` está ativo, a fonte de clock do PC é comutada para o sinal `clk_manual`, o qual é acionado manualmente pelo usuário por meio do pressionamento de um botão. Assim, ao pressionar o botão, um pulso de clock é gerado, permitindo que o PC avance para a próxima instrução, retomando seu funcionamento normal em seguida.

4.4.3 Memória de Instrução

O módulo `instruction_ROM`, presente no Apêndice E, implementa a memória de instruções do processador, responsável por armazenar e fornecer as instruções do programa a ser executado. Trata-se de uma memória somente leitura (ROM), implementada com base no *template* de memória ROM fornecido pelo *software* Quartus, de modo a aproveitar a memória física já presente no FPGA. Essa memória opera de forma síncrona, sendo acessada a cada ciclo de *clock* para disponibilizar a instrução localizada no endereço fornecido pelo contador de programa. O sinal de *clock* utilizado na entrada desse módulo

é específico para leitura, operando em uma frequência significativamente superior à do processador, de forma a não introduzir atrasos no funcionamento do caminho de dados da arquitetura.

A memória é parametrizável tanto em relação à largura do endereço (`ADDR_WIDTH`) quanto ao arquivo de inicialização (`init_file`), que define o conteúdo inicial da ROM. Por padrão, utiliza-se o arquivo `rom_init.mif`, que contém as instruções codificadas em linguagem binária. No bloco `initial`, o conteúdo desse arquivo é carregado no vetor `rom` por meio da diretiva `$readmemb`, que interpreta o arquivo como uma sequência binária, armazenando uma instrução de 32 bits em cada posição da memória. O acesso à memória ocorre na borda de subida do sinal de *clock* (`clk`), momento em que a instrução correspondente ao endereço `addr` é colocada na saída `inst_code`.

Após a instrução ser obtida, seu código é desmembrado no caminho de dados do processador. Os diferentes campos da instrução são direcionados para os módulos correspondentes: o campo de `opcode` para a unidade de controle, os endereços para leitura no banco de registradores, o campo de deslocamento (`shamt`) para a ALU em operações de formato R, como `sll` e `srl`, e o campo imediato para o módulo de extensão de sinal, utilizado em instruções de formato I.

Embora a arquitetura do processador opere conceitualmente com memória de instruções no formato *word addressable*, o que permitiria uma extensão de até 2^{32} palavras (equivalente a cerca de 4 bilhões de instruções distintas), o módulo foi implementado com um espaço de endereçamento reduzido de 2^8 palavras, totalizando 256 instruções. Essa limitação foi adotada intencionalmente para simplificar o projeto, tornando-o mais leve e adequado para testes e depuração em ambiente educacional, sem comprometer o funcionamento básico da arquitetura.

4.4.4 Unidade de controle

A unidade de controle foi implementada no módulo `control_unit`, apresentada no Apêndice F, e opera diretamente a partir da instrução acessada pela memória de instruções. Em particular, a entrada `inst_op` recebe o campo de opcode da instrução e com base nele determina os sinais de saída conforme estabelecido na Tabela 9.

4.4.5 Extensor de sinal

O módulo `sign_extension`, apresentado no Apêndice G, tem como objetivo estender um valor imediato de 16 bits para 32 bits com preservação do sinal, conforme especificado pela arquitetura MIPS. Esse procedimento é fundamental para permitir operações aritméticas e lógicas com operandos imediatos em instruções do tipo I ou para o armazenamento correto da entrada com os *switches* do kit FPGA, mantendo o tamanho

de dados utilizado na arquitetura.

A entrada do módulo é `inst_imm`, um vetor de 16 bits que representa o campo imediato da instrução. A saída é `inst_imm_extended`, um vetor de 32 bits com o valor estendido. A extensão de sinal é feita replicando o bit mais significativo (bit 15) de `inst_imm` nas 16 posições mais altas de `inst_imm_extended`. Essa operação garante que, caso o número imediato original seja negativo (bit 15 igual a 1), os 16 bits superiores também serão 1, mantendo o valor correto no formato de 32 bits. Se for positivo (bit 15 igual a 0), os bits superiores serão 0.

4.4.6 Banco de registradores

O banco de registradores, implementado conforme o Apêndice H, é uma estrutura fundamental da arquitetura MIPS que armazena temporariamente dados e operandos utilizados pelas instruções durante a execução. Este banco contém 32 registradores de 32 bits cada, numerados de 0 a 31, sendo que o registrador de índice zero ($zero) é fixo com valor constante zero, conforme especificação da arquitetura, e não pode ser modificado. Para garantir essa característica, a lógica de escrita do banco verifica se o endereço de destino da escrita é diferente de zero; caso contrário, nenhuma alteração é realizada, preservando o valor constante deste registrador.

A escrita nos registradores ocorre na borda positiva do *clock*, controlada pelo sinal `regwrite`, que habilita a operação apenas quando ativo. Caso o endereço de escrita seja diferente de zero, o dado de entrada `write_data` é armazenado no registrador correspondente, garantindo a integridade e sincronização da operação.

Por outro lado, a leitura dos registradores é realizada de forma combinacional, ou seja, a partir dos endereços fornecidos pelos campos da instrução, os dados dos registradores são disponibilizados imediatamente nas saídas `reg1_data` e `reg2_data`, sem a necessidade de sincronização com o *clock*. Isso permite que as operações aritméticas, lógicas e de controle tenham acesso rápido aos operandos necessários durante o ciclo de execução da instrução. Quando os endereços de leitura apontam para o registrador zero, a saída é forçada para zero, respeitando a convenção da arquitetura e garantindo que este registrador funcione sempre como uma constante.

O módulo `mux_write_reg`, do Apêndice I, complementa o banco de registradores ao controlar dinamicamente qual endereço de registrador será utilizado para a operação de escrita. Por meio de um multiplexador controlado pelo sinal `regdst`, que possui três bits, ele escolhe entre três possíveis destinos: o campo `rt` (bits 20 a 16 da instrução), o campo `rd` (bits 15 a 11) ou o registrador número 31. Esta seleção é fundamental para suportar os diferentes formatos de instruções da arquitetura MIPS, que podem usar distintos campos para especificar o registrador destino dependendo do tipo da instrução (R, I ou J). Por

exemplo, instruções do tipo R geralmente utilizam o campo `rd`, enquanto instruções do tipo I usam `rt`, e instruções de salto e link usam o registrador 31 para salvar o endereço de retorno.

4.4.7 Unidade Lógica e Aritmética

A Unidade Lógica e Aritmética é responsável por realizar todas as operações aritméticas e lógicas necessárias para a execução das instruções. No projeto apresentado, a ALU opera com operandos de 32 bits, compatíveis com a arquitetura MIPS que utiliza valores de tamanho fixo para garantir uniformidade no processamento. O módulo `ALU`, mostrado no Apêndice J, recebe dois operandos de entrada, `X` e `Y`, um valor de deslocamento (`shamt`) para operações de *shift*, e um código de operação (`op`) que determina qual função será executada.

As operações suportadas incluem adição, subtração, multiplicação, divisão, operações lógicas (`AND`, `OR`, `XOR`, `NOR`) e deslocamentos lógicos à esquerda (`SLL`) e à direita (`SRL`), além da operação `LUI`, que carrega o imediato na parte alta do registrador. A execução dessas operações ocorre de forma combinacional, garantindo que a saída `alu_out` seja atualizada imediatamente com o resultado da operação especificada. Além do resultado, a ALU gera sinais auxiliares: o sinal `zero` indica quando o resultado é zero e o sinal `less` indica o bit mais significativo do resultado, tendo utilidade prática para a operação de subtração usada na comparação entre os valores de dois registradores. Ambos os sinais contribuem para as decisões de desvio que serão processadas posteriormente na Unidade Lógica de *Branch*.

Para executar essas operações, o primeiro dos operandos sempre será o dado lido do registrador do campo "`rs`" da instrução. No entanto, para definir o segundo operando, é necessária a seleção entre o dado do outro registrador lido ou o imediato estendido, conforme a necessidade da operação em execução no ciclo atual. Para isso, o projeto utiliza um multiplexador genérico de 1 bit, que recebe dois dados de 32 bits e um sinal de controle, e escolhe qual dos dois será encaminhado como saída. Esse multiplexador, descrito no módulo `mux_2to1` do Apêndice K, compara o sinal `flag` — que neste caso corresponde ao sinal de controle `ALUSrc` — e, com base em seu valor, seleciona entre o conteúdo do registrador `rt` ou um valor imediato estendido. Quando `ALUSrc` é igual a 0, o valor utilizado como segundo operando da ALU será o do registrador `rt`, o que ocorre normalmente em instruções do tipo R. Quando `ALUSrc` é igual a 1, o operando selecionado é o valor imediato, como necessário em instruções do tipo I. Essa lógica de seleção torna a ALU versátil e adaptável às diferentes instruções da arquitetura MIPS, garantindo que ela opere corretamente independentemente do formato da instrução em execução.

4.4.7.1 Unidade de Controle da ALU

O módulo `ALU_control`, apresentado no Apêndice L, é responsável por traduzir os códigos de operação da instrução (`ALUop`) e, quando necessário, o campo função (`inst_funct`) para o código específico da operação a ser realizada pela ALU. Esse controle é essencial para modularizar o projeto e separar a lógica de decodificação da operação da implementação da ALU propriamente dita. Ele também gera o sinal `jumpreg`, que indica a necessidade de realizar um salto baseado no registrador (Jump Register).

O controle funciona por meio de um `case` que interpreta o código `ALUop`, associado a cada tipo de instrução, definindo operações padrão como soma, `AND`, `OR`, `XOR`, subtração e `LUI`. Para instruções do formato R (`ALUop = 3'b111`), o módulo examina o campo função da instrução para identificar a operação exata, incluindo multiplicação, divisão, *shifts* e o salto via registrador. O sinal `jumpreg` é ativado especificamente quando a operação é `Jump Register`, sinalizando para os módulos de controle e atualização de `PC` que o próximo endereço de execução será obtido do registrador em vez do fluxo sequencial normal.

Dessa forma, o `ALU_control` atua como um decodificador especializado, traduzindo as instruções em comandos claros e específicos para a ALU, garantindo que o caminho de dados execute corretamente as operações previstas pelo conjunto de instruções do processador MIPS monociclo.

4.4.8 Unidade Lógica de *Branch*

A `Unidade Lógica de Branch` (`branch_logic_unit` no Apêndice M), a partir do sinal de controle "BranchType", é responsável por decidir se um desvio condicional deve ser tomado com base nos resultados produzidos pela ALU. Essa unidade recebe sinais indicativos do resultado da última operação da ALU, como o sinal zero (`alu_zero`), que indica se o resultado foi zero, e o sinal less (`alu_less`), que indica se o resultado foi negativo, além de um código que especifica o tipo de desvio a ser avaliado (`branch_type`).

O módulo interpreta o tipo de *branch* solicitado entre várias condições típicas do conjunto de instruções MIPS, como `BEQ` (*branch if equal*), `BNE` (*branch if not equal*), `BGT` (*branch if greater than*), `BLT` (*branch if less than*), `BGE` (*branch if greater or equal*) e `BLE` (*branch if less or equal*). Com base nesses parâmetros, a unidade realiza comparações lógicas entre os sinais da ALU para determinar se a condição do *branch* foi satisfeita.

Por exemplo, para `BEQ`, o desvio será tomado se o resultado da ALU for zero, indicando igualdade entre operandos; para `BNE`, o *branch* ocorre quando o resultado não é zero.

Essa lógica combinacional é implementada em um bloco `always_comb`, garantindo que a saída `branch` seja atualizada imediatamente quando qualquer uma das entradas mudar, permitindo decisões rápidas de controle de fluxo sem a necessidade de sincronismo

por *clock*. O sinal de saída `branch` é então utilizado pelo controlador de fluxo para alterar o valor do contador de programa (PC), possibilitando o salto para o endereço destino da instrução de desvio, caso a condição seja verdadeira.

Assim, a Unidade Lógica de *Branch* integra-se ao caminho de dados e controle do processador, permitindo a execução correta das instruções de desvio condicional, fundamentais para o funcionamento de estruturas de controle como *loops* e decisões condicionais dentro do código executado pelo processador MIPS monociclo.

4.4.9 Memória de dados

A memória de dados (`data_RAM` no Apêndice N) é responsável por armazenar e fornecer os dados que são lidos e escritos durante a execução do programa. Diferente da memória de instruções, essa memória é de leitura e escrita (RAM) e opera com acesso a palavras de 32 bits, compatível com a arquitetura MIPS monociclo de 32 bits, que trabalha com dados de tamanho fixo.

A arquitetura MIPS de 32 bits suporta, em teoria, até 2^{32} endereços distintos, o que corresponderia a 4GB de espaço de endereçamento de memória. No entanto, por uma questão de viabilidade prática e limitação de recursos em implementações com FPGA, optou-se por um tamanho reduzido: o módulo implementa uma memória de 4KB, organizada em 1024 palavras de 32 bits, acessadas por endereços no formato *word-addressable*. O endereço de 32 bits recebido é indexado pelos 10 bits menos significativos (`addr[9:0]`), limitando o acesso à faixa de endereços entre 0 e 1023, o que simplifica o projeto e mantém a memória adequada para fins educacionais e testes.

A respeito da escrita na memória, ela é realizada de forma síncrona, ocorrendo na borda de subida do sinal de *clock* específico para escrita, denominado `write_clk` — o mesmo utilizado no ciclo geral de funcionamento do processador —, quando o sinal de controle `memwrite` está ativo. Nesse instante, o dado fornecido na entrada `data` é armazenado na posição de memória indicada pelo endereço. Essa sincronização garante que as escritas aconteçam de forma ordenada e consistente com o ritmo do processador.

Já a leitura é também realizada de forma síncrona, ocorrendo na borda de subida do sinal de *clock* dedicado à leitura, denominado `read_clk`. Enquanto o sinal `memread` estiver ativo, o conteúdo da posição de memória endereçada é disponibilizado na saída `mem_out` após a borda do *clock* de leitura.

Embora, no contexto deste projeto, o uso de um sinal de controle para leitura não seja estritamente necessário, sua presença se justifica por alinhamento com arquiteturas reais, como a MIPS. Nessas arquiteturas, o acesso à memória pode ser uma operação custosa em termos de desempenho, tornando essencial o controle explícito sobre a leitura.

Dessa forma, a memória de dados atua como o espaço principal para armazenamento

temporário de variáveis, operandos intermediários e resultados, integrando-se ao caminho de dados do processador para suportar instruções que envolvem acesso à memória, como *loads* e *stores*.

4.4.10 Seleção de dados de escrita/saída

O módulo `mux_write_back`, apresentado no Apêndice I, é responsável por selecionar a fonte de dados que será encaminhada como saída final do processador, podendo ser usada para escrita no banco de registradores ou para envio direto à saída da arquitetura, dependendo da instrução em execução. Essa seleção é feita com base no sinal de controle `write_back`, que determina qual das cinco fontes disponíveis será escolhida.

As entradas do multiplexador incluem: o dado proveniente da memória (`mem_data`), utilizado em instruções de carregamento como `lw`; o resultado da ALU (`alu_data`), comum em instruções aritméticas e lógicas como `add`, `sub`, `addi`; o valor do PC incrementado (`pc_plus_one`), utilizado por instruções de salto como `jal`; um valor lido dos *switches* da placa (`input_data`), utilizado na instrução de *input*; e o conteúdo do registrador `rs` (`rs_data`), que pode ser usado tanto para operações internas quanto para instruções de saída, como a instrução `OUT`.

Nesse último caso, quando a instrução `OUT` é executada, a seleção do dado vindo de `rs_data` permite que o valor contido nesse registrador seja diretamente enviado para o módulo de saída, e não necessariamente para escrita no banco de registradores. Assim, o `mux_write_back` assume um papel mais amplo no *datapath*, funcionando como ponto final da instrução antes do encaminhamento do valor selecionado para seu destino adequado — seja ele um registrador interno ou uma interface de saída.

Todo o comportamento do módulo é implementado de forma combinacional, permitindo que a saída `mux_out` reflita imediatamente qualquer alteração nas entradas ou no controle `write_back`, assegurando eficiência na propagação de dados durante a fase de *write-back* ou na execução de instruções de saída.

4.4.11 Módulo de Atualização do *Program Counter*

O módulo `pc_update_logic`, apresentado no Apêndice D implementa a lógica de atualização do contador de programa, englobando os três multiplexadores presentes no caminho de dados, conforme ilustrado na Figura 7. Esses multiplexadores são responsáveis pela seleção do próximo valor do PC com base nos sinais de controle `jumpreg`, `branch` e `jump`.

O primeiro multiplexador seleciona entre o valor sequencial PC + 1 e o conteúdo do registrador (`reg1_data`), controlado pelo sinal `jumpreg`, caracterizando instruções como `JR`. O segundo multiplexador, controlado pelo sinal `branch`, decide entre manter a execução

sequencial ou redirecionar para o endereço de desvio condicional, calculado como PC + 1 + `inst_offset`. O terceiro multiplexador realiza a seleção final entre o resultado anterior e o endereço de salto absoluto, construído a partir da concatenação dos bits mais significativos de pc com o campo `inst_target`; esse multiplexador é controlado pelo sinal `jump`.

A correta propagação dos sinais de controle garante que o valor de `pc_next` seja atualizado adequadamente conforme o tipo de instrução executada, respeitando o funcionamento previsto para saltos, desvios e execução sequencial dentro da arquitetura MIPS. Assim, esse valor de saída será utilizado para a atualização do módulo `program_counter`, mostrado anteriormente, que atualizará o PC no próximo sinal de *clock*.

4.4.12 Topo do sistema

O módulo `system_top` (Apêndice A) representa o nível mais alto de integração do sistema, sendo responsável por conectar a lógica do processador à interface física do FPGA. Sua função principal é encapsular a execução do processador, gerenciar os sinais de entrada e saída do kit e direcionar os dados adequadamente entre os módulos internos e os dispositivos externos.

As entradas desse módulo incluem os sinais de `clk` e `read_clk`, utilizados respectivamente para a lógica do sistema e para as leituras síncronas das memórias. O sinal `reset` é usado para reinicialização do sistema, enquanto `btn_io` corresponde ao botão físico utilizado em operações de entrada/saída. O vetor `switches`, com 16 bits, fornece os dados de entrada externos ao processador, e o sinal `sw_negative` indica se esse valor deve ser interpretado como negativo.

As saídas do módulo são conectadas aos dispositivos periféricos do FPGA, responsáveis por exibir o estado e os dados do processador durante a execução. Entre esses sinais estão o `input_led` e o `output_led`, que indicam os momentos de espera por interação do usuário nas instruções de *input* e *output*, respectivamente, e o `halt_led`, que sinaliza a paralisação do sistema. Os dados processados são continuamente apresentados em formato binário nos 18 LEDs da placa, por meio da saída `data_led`, permitindo acompanhar o funcionamento do processador a cada instrução. Já os resultados das instruções OUT são exibidos também nos *displays* de sete segmentos, conectados às saídas `HEX6` até `HEX0`, servindo como principal meio de visualização da saída final ao usuário.

Internamente, o módulo é composto por quatro submódulos principais: o `mips_processor`, responsável pela lógica central do processador; o `display_output`, que converte os dados binários em sinais apropriados para os displays; o `clock_divider`, que ajusta a frequência do *clock* para o processador;e o `button_debouncer`, que filtra oscilações do botão físico.

Além dos submódulos, o `system_top` inclui duas lógicas combinacionais auxiliares. A primeira delas, representada pelo sinal `bin_value_mux`, realiza a seleção do valor a

ser exibido nos *displays* de sete segmentos: quando o sistema está em uma operação de entrada (sinalizada por `flag_in`), o valor exibido é aquele configurado pelos *switches*; caso contrário, exibe-se o valor de saída gerado pelo processador. A segunda lógica define o sinal `flag_io_negative`, que determina se o número exibido deve apresentar o sinal negativo. Em operações de entrada, essa decisão é baseada diretamente no estado do *switch* `sw_negative`, que é o comando do usuário de inserção de um valor negativo; já em operações de saída, o controle é feito pela `flag_negative`, fornecida pelo próprio processador a partir do valor de saída.

4.4.13 Divisor de Clock

O módulo `clock_divider`, apresentado no Apêndice P, tem como função gerar um sinal de *clock* com frequência reduzida a partir do *clock* principal do sistema, sendo utilizado para desacelerar o funcionamento do processador e, assim, facilitar a observação e depuração de sua execução, principalmente em um contexto educacional. Ele opera por meio de um contador que alterna o sinal de saída entre alto e baixo após atingir metade do valor definido pelo parâmetro `DIVISOR`, dividindo, portanto, a frequência de entrada por esse fator.

A frequência final gerada é dada pela razão entre a frequência do *clock* de entrada e o valor do divisor. Por exemplo, com um *clock* de 50 MHz e `DIVISOR = 28'd50_000_000`, obtém-se uma frequência de saída de 10 Hz.

O código base do módulo foi adaptado do projeto disponível do site FPGA4Students (9), ajustando-o às necessidades específicas do projeto aqui descrito.

4.4.14 Debouncer

O módulo `button_debouncer`, presente no Apêndice Q, tem como objetivo eliminar os *bouncings* presentes na leitura de botões físicos, que podem causar múltiplas detecções de um mesmo acionamento devido à instabilidade mecânica do contato elétrico. Esse tipo de ruído pode comprometer o correto funcionamento do sistema, especialmente em aplicações que dependem de transições precisas. Essa filtragem será aplicada no botão determinado para a confirmação de entrada e saída do usuário.

A estratégia adotada nesse projeto foi adaptada do código disponibilizado pelo site FPGA4Students (10). A principal característica dessa abordagem é que ela evita a criação de um novo domínio de *clock*, utilizando em vez disso um sinal de habilitação (*clock enable*) para controlar a amostragem lenta do botão.

O módulo é composto por três flip-flops conectados em série, controlados por esse sinal de habilitação. A lógica combinacional da saída garante que apenas uma transição

válida do botão (pressionamento ou liberação) seja reconhecida, gerando um pulso único e estável para cada evento.

4.4.15 Saída para Display de Sete Segmentos

O módulo `display_output`, apresentado no Apêndice T, é responsável por realizar a conversão e a exibição dos dados binários presentes na saída do processador ou na entrada proveniente dos *switches* para os displays de sete segmentos do kit FPGA. Seu funcionamento depende dos sinais de controle `flag_in` e `flag_out`, que determinam quando os valores devem ser apresentados, além do sinal `flag_negative`, que indica a necessidade de exibir um número negativo.

A entrada principal de dados do módulo é o vetor binário de 20 bits denominado `bin_value`, que pode conter tanto o valor inserido externamente por meio dos *switches* quanto o valor de saída proveniente de um registrador do processador. No contexto de uma instrução de entrada, o usuário fornece um valor em módulo direto, e eventuais conversões para complemento de dois — necessárias para o armazenamento interno no processador — são realizadas posteriormente pela lógica do sistema. Dessa forma, a conversão de binário para BCD realizada neste módulo considera diretamente o valor em módulo, sem levar em conta o sinal.

Já no caso das instruções de saída, o valor fornecido pelo processador encontra-se em complemento de dois, o que exige sua conversão prévia para módulo antes da conversão para BCD. Essa transformação é feita internamente no módulo `display_output` sempre que o sinal `flag_negative` estiver ativo juntamente com `flag_out`. A sinalização do sinal de negativo no *display*, por sua vez, já é previamente determinada pelo módulo `system_top`, conforme descrito anteriormente.

Para exibição nos displays, o valor binário é convertido para o formato BCD (*Binary-Coded Decimal*) através do módulo auxiliar `bin_to_bcd` (Apêndice U), que aplica o algoritmo *Double Dabble*, resultando nos dígitos separados em centenas de milhar, dezenas de milhar, milhares, centenas, dezenas e unidades. Esses dígitos são então convertidos para o padrão dos displays de sete segmentos por meio da função interna `bcd_to_7seg`, que realiza o mapeamento de cada dígito decimal para seu respectivo código de ativação dos segmentos.

Por fim, o módulo controla sete displays individuais (`HEX6` a `HEX0`), sendo o mais significativo (`HEX6`) reservado para exibir o sinal negativo, caso necessário. A exibição só é ativada quando ao menos uma das flags `flag_in` ou `flag_out` está em nível lógico alto, garantindo que o conteúdo do display reflita apenas momentos de entrada ou saída de dados.

4.5 Pinagem do kit FPGA

Conforme definido pelo módulo do topo do sistema, a interface entre o processador e o kit FPGA será composta pelo uso dos *clocks*, *switches*, botões, LEDs e *displays* disponibilizados na placa. As entradas incluem o botão físico `btn_io`, conectado ao pino `PIN_M23` (KEY0), usado para controle de operações de entrada e saída, além dos *switches* que formam um vetor de 16 bits para entrada de dados, numerados de 0 a 15. Também são utilizados os sinais `reset` e `sw_negative`, correspondentes aos switches 17 e 16, localizados nos pinos `PIN_Y23` e `PIN_Y24`, respectivamente, que controlam a reinicialização do sistema e a indicação de valores negativos na entrada. Para sincronização, são empregados dois clocks: `clk`, no pino `PIN_Y2`, e `read_clk`, no pino `PIN_AG14`, ambos de 50 MHz.

Nas saídas, utilizaram-se os LEDs `halt_led`, `input_led` e `output_led`, ligados aos pinos `PIN_E21` (LEDG[0]), `PIN_G21` (LEDG[7]) e `PIN_G22` (LEDG[6]), que indicam, respectivamente, a parada do processador e as esperas por interação do usuário. Além disso, para a exibição da instrução de saída, são usados os *displays* de sete segmentos, identificados de `HEX0` (LSB) a `HEX5` (MSB), com o `HEX6` destinado a mostrar o sinal de negativo. Também como saída, todos os LEDs vermelhos (LEDR[0] ao LEDR[17]) foram utilizados para representar o dado em questão em seu valor binário.

4.6 Algoritmo de validação

Conforme estabelecido nos objetivos geral e específicos deste projeto, o seu propósito principal é projetar e implementar um processador em um kit FPGA capaz de executar algoritmos básicos de lógica de programação. Para validar essa funcionalidade, foi desenvolvido um algoritmo que simula a construção de um vetor na memória e a posterior análise dos seus elementos.

O funcionamento do algoritmo consiste na inserção de uma quantidade N de elementos na memória de dados, sendo esse valor fornecido pelo usuário — desde que não ultrapasse os limites da memória disponível. Os dados são inseridos sequencialmente, simulando a criação de um vetor.

Após a inserção, o processador percorre sequencialmente a memória e realiza comparações entre os elementos armazenados. Durante esse processo, ele identifica e armazena, em registradores distintos, o menor e o maior valor encontrados. Ao final da análise, os resultados são exibidos: primeiro o maior valor, seguido do menor.

O código desenvolvido pode ser visualizado e interpretado conforme o Código 1, em que segue a sintaxe da linguagem Assembly do MIPS32 tradicional, com a adição de duas instruções específicas: `IN $rt` e `OUT $rt, $rs`, destinadas às operações de entrada e saída, respectivamente.

Código 1 – Algoritmo de validação

```
1     in $s0 #Total de elementos a serem inseridos
2     addi $t6, $zero, 1023
3     ble $s0, $zero, HALT
4     bge $s0, $t6, HALT
5     add $a0, $s0, $zero
6     jal READ_INPUTS #Leitura de entradas e escrita na memoria
7
8     add $t1, $zero, $zero
9     add $t2, $zero, $zero
10    lw $t3, 0($t2)
11    add $t4, $t3, zero #Inicializa $t4 como maior valor
12    add $t5, $t3, $zero #Inicializa $t5 como menor valor
13
14 FIND_LOOP:
15    beq $t1, $s0, PRINT_RESULT #Loop termina apos processar N
          elementos
16    lw $t3, 0($t2)
17    ble $t3, $t4, CHECK_MIN #Caso nao seja o maior elemento,
          salta para a verificacao de menor elemento sem atualizar
          $t4
18    add $t4, $t3, $zero
19
20 CHECK_MIN:
21    bge $t3, $t5, NEXT_ELEMENT #Atualiza menor valor, se
          necessario
22    add $t5, $t3, $zero
23
24 NEXT_ELEMENT:
25    addi $t1, $t1, 1
26    addi $t2, $t2, 1
27    j FIND_LOOP #Continua loop de leitura do vetor
28
29 PRINT_RESULT:
30    out $t4 #Imprime o maior valor
31    out $t5 #Imprime o menor valor
32 HALT:
33    halt
34
35 READ_INPUTS:
36    add $t1, $zero, $zero
37    add $t2, $zero, $zero
```

```
38
39 INPUT_LOOP:
40     beq $t1, $a0, END_READ #Loop de leitura termina apos N
           entradas
41     in $v0
42     sw $v0, 0($t2) #Armazena elemento lido na memoria
43     addi $t1, $t1, 1
44     addi $t2, $t2, 1
45     j INPUT_LOOP
46
47 END_READ:
48     jr $ra #Retorna para o ponto apos o jal
```

Fonte: O Autor

Esse algoritmo foi escolhido por permitir uma validação abrangente das funcionalidades essenciais do processador. Ele envolve múltiplos acessos à memória por meio das instruções `LW` (leitura) e `SW` (escrita), bem como operações de entrada e saída via `IN` e `OUT`. Além disso, são exercitadas instruções de desvio incondicional (`J`, `JAL`, `JR`) e condicional (`BEQ`, `BGE`, `BLE`), além de instruções aritméticas como `ADD` (soma entre registradores) e `ADDI` (soma com imediato). Isso garante uma cobertura completa dos principais blocos funcionais do processador.

5 Resultados Obtidos e Discussões

Os resultados obtidos com a implementação do processador baseado na arquitetura MIPS 32 são apresentados neste capítulo e estão organizados em duas etapas complementares: a simulação da Unidade de Processamento no ambiente *ModelSim* e a validação prática do sistema computacional a partir da execução de um algoritmo de lógica de programação no kit FPGA.

Na primeira etapa, a simulação no ambiente *ModelSim* permitiu verificar o funcionamento dos módulos individuais da Unidade de Processamento, assegurando a consistência do projeto antes da síntese no hardware. Já na segunda, integrou-se a Unidade de Processamento, Unidade de Controle e a Interface do Sistema ao kit FPGA, conectando também os dispositivos físicos como *switches*, botões, LEDs e *displays*. Com isso, foi possível testar a execução de um algoritmo de lógica de programação, observando diretamente as operações de entrada, processamento e saída.

Os resultados obtidos ao longo dessas duas fases são discutidos a seguir, destacando o funcionamento correto do sistema.

5.1 Simulação dos módulos da Unidade de Processamento

5.1.1 Simulando o *Program Counter*

Iniciando pelo *Program Counter*, foi analisado o funcionamento do seu módulo em conjunto com o módulo `pc_update_logic`. A simulação considera as seguintes variáveis: o sinal de `reset`, que zera o valor do PC; o `pc_current_addr`, que armazena o endereço atual; as variáveis de dados `inst_target` (campo "*target*"de instruções do tipo J), `inst_offset` (imediato já estendido utilizado em instruções de desvio), e `reg1_data`, que representa o valor lido do registrador "rs". Além disso, são utilizadas as *flags* `branch`, `jump`, `jumpreg` e `halt`, juntamente com o sinal de *clock*, que promove o funcionamento síncrono do módulo `program_counter`. Por fim, o dado `pc_next_addr`, calculado pela `pc_update_logic`, representa o próximo endereço a ser atribuído ao PC.

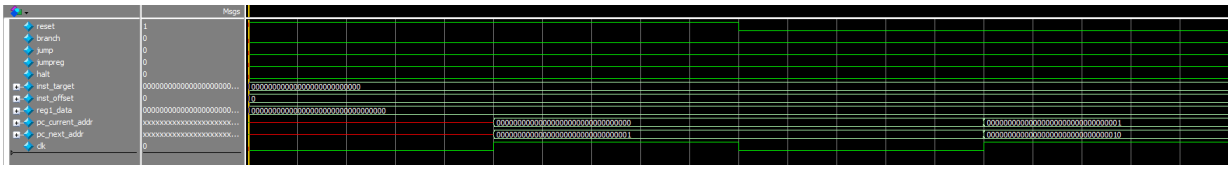
No que se refere à conexão entre os módulos, o `pc_update_logic` recebe como entrada as flags de controle, o `pc_current_addr` e as variáveis de dados. Com base nessas informações, o endereço `pc_next_addr` é calculado e passado como entrada para o `program_counter`, que, na borda de subida do *clock*, atualiza o valor de `pc_current_addr`. Esse módulo também responde aos sinais de `reset` e `halt`, como mencionado.

Também, vale ressaltar que o módulo `pc_update_logic`, para determinar o próximo

valor do contador de programa com base nas flags de controle `branch`, `jump` e `jumpreg`, é utilizado o módulo auxiliar `mux_2to1`, que realiza a escolha entre os endereços possíveis de forma condicional. Dessa maneira, o correto funcionamento do módulo `pc_update_logic` está diretamente ligado ao comportamento esperado dos multiplexadores. Assim, uma vez validada a simulação do comportamento do Program Counter, também se confirma o funcionamento adequado dos multiplexadores envolvidos nesse processo.

O primeiro teste de funcionamento do *Program Counter*, apresentado na Figura 8, inicia com o sinal de `reset` ativo e todas as demais entradas zeradas. Observa-se que, após a primeira borda de subida do *clock*, o valor do PC é definido como zero, e a lógica combinacional do `pc_update_logic` já calcula o próximo endereço como `PC + 1`. Nas subidas seguintes do *clock*, esse processo se repete corretamente.

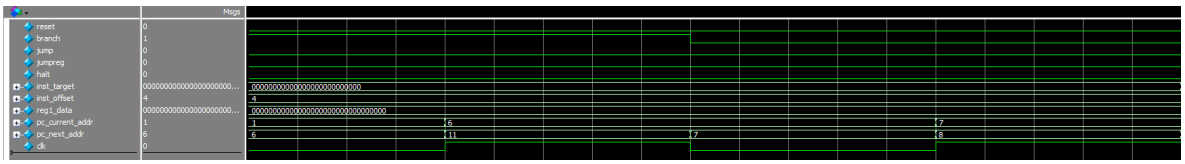
Figura 8 – Simulação do *Program Counter* com sinais de controle inativos



Fonte: O Autor

Ao ativar a flag `branch`, como ilustrado na Figura 9, o comportamento do PC muda: agora o próximo endereço passa a ser calculado como `PC + 1 + offset`. Para esse teste, o valor 4 foi atribuído ao campo `inst_offset`, apresentado em decimal. A cada borda de subida do *clock*, verifica-se que a operação é corretamente executada, com o PC atualizado conforme esperado, e o valor de `pc_next_addr` refletindo adequadamente a lógica de desvio. Quando o sinal `branch` é desativado, o PC volta ao comportamento padrão de incremento sequencial.

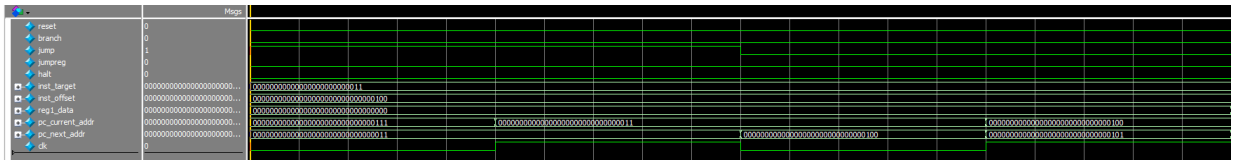
Figura 9 – Simulação do *Program Counter* com sinal `Branch` ativo



Fonte: O Autor

Na simulação com a flag `jump` ativa (Figura 10), o valor de `pc_next_addr` passa a ser calculado a partir da concatenação dos bits superiores do `pc_current_addr` com o valor de `inst_target`. No instante em que o sinal é ativado, observa-se que o endereço de salto é corretamente formado e atribuído ao PC na borda seguinte do *clock*. Após a desativação da flag `jump`, o PC retorna ao fluxo sequencial de execução.

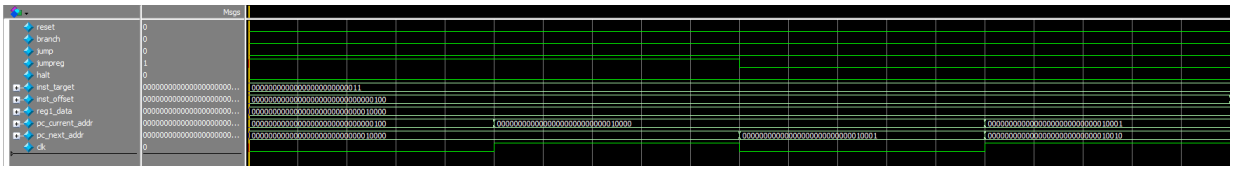
Figura 10 – Simulação do *Program Counter* com sinal `Jump` ativo



Fonte: O Autor

Com o sinal `jumpreg` ativo, a lógica altera o PC para o valor contido em `reg1_data`, normalmente utilizado em instruções de retorno como `jr $ra`. A Figura 11 mostra que, na borda de subida do *clock* subsequente à ativação da flag, o PC é atualizado com o valor lido do registrador. Assim como nos outros casos, ao desativar o sinal, o funcionamento normal do PC é retomado.

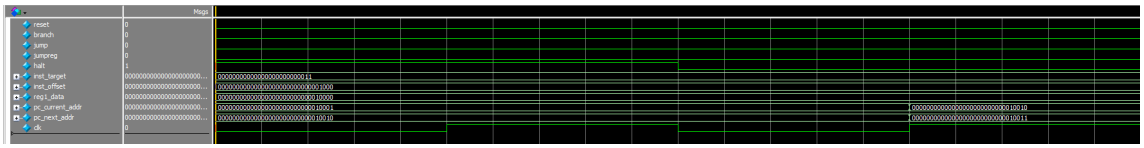
Figura 11 – Simulação do *Program Counter* com sinal `JumpReg` ativo



Fonte: O Autor

Finalmente, o comportamento do sistema com a flag `halt` ativa é apresentado na Figura 12. Nesse caso, observa-se que o valor do PC permanece inalterado, mesmo após a borda de subida do *clock*. A atualização do PC só volta a ocorrer após a desativação do sinal `halt`, retomando o fluxo normal de execução.

Figura 12 – Simulação do *Program Counter* com sinal `Halt` ativo



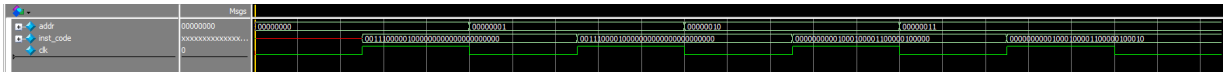
Fonte: O Autor

5.1.2 Simulando a Memória de Instrução

A simulação da memória de instruções, apresentada na Figura 13, utiliza como entradas o sinal `addr` e como saída o sinal `inst_code`. Para cada valor atribuído a `addr`, ocorre o acesso à posição correspondente da memória, retornando o código da instrução armazenado naquele endereço. Todos os códigos foram previamente definidos por meio de um arquivo `.mif` (do formato *Memory Initialization File*), responsável por inicializar o conteúdo da memória. O processo de leitura é sequencial e depende da borda de subida

do *clock*, garantindo o comportamento síncrono do sistema. Observa-se, portanto, que a simulação apresenta um funcionamento coerente e conforme o esperado.

Figura 13 – Simulação da memória de instruções

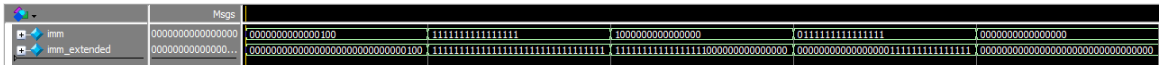


Fonte: O Autor

5.1.3 Simulando o Extensor de Sinal

O teste do módulo de extensão de sinal, com resultado na Figura 14, utiliza como entrada o campo `imm` e gera como saída `imm_extended`. Para cada valor testado, observou-se que o sinal foi corretamente preservado e estendido de 16 para 32 bits, validando o funcionamento do módulo.

Figura 14 – Simulação do extensor de sinal



Fonte: O Autor

5.1.4 Simulando o Banco de Registradores

A simulação do banco de registradores, ilustrada na Figura 15, conta com as entradas `reg1_addr` e `reg2_addr`, que determinam os endereços dos registradores a serem lidos, além de `write_reg_addr`, que indica o endereço do registrador de escrita. O sinal `regwrite` controla se a escrita no banco de registradores deve ser realizada. As saídas `reg1_data` e `reg2_data` retornam os dados lidos nos respectivos registradores especificados.

Figura 15 – Simulação do banco de registradores



Fonte: O Autor

No teste realizado para o módulo `register_file`, inicialmente, todos os valores se encontram zerados. Nesse momento, os sinais `reg1_data` e `reg2_data` retornam o valor 0, já que os endereços acessados correspondem ao registrador de número zero ($zero), que por convenção da arquitetura MIPS, sempre armazena o valor zero.

Com o sinal `regwrite` ativado, foram realizadas três operações de escrita com valores e endereços distintos, incluindo um teste com escrita no endereço zero. As escritas

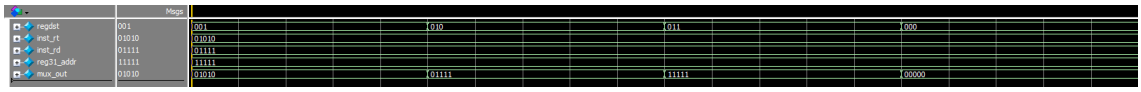
ocorrem a cada borda de subida do *clock*. Após as escritas, o sinal `regwrite` é desativado, e passam a ser realizadas leituras instantâneas nos registradores, com os endereços de leitura variando a cada pulso de *clock*.

Observa-se que os valores previamente escritos nos dois primeiros registradores são corretamente lidos. No entanto, no caso do registrador de número zero, mesmo que uma tentativa de escrita tenha sido realizada, o valor retornado continua sendo zero, confirmando a proteção contra escrita nesse registrador, conforme esperado na arquitetura MIPS.

5.1.5 Multiplexador para seleção de registrador de escrita

Para facilitar a visualização do funcionamento do multiplexador responsável pela seleção do registrador de escrita, sua simulação foi realizada separadamente do banco de registradores. A forma de onda correspondente está apresentada na Figura 16.

Figura 16 – Simulação do banco de registradores



Fonte: O Autor

Neste teste, as entradas fornecidas ao multiplexador são: `inst_rt` e `inst_rd`, que correspondem aos campos de endereço de registradores em instruções do tipo I e R, respectivamente, além de `reg31_addr`, que contém o endereço fixo do registrador de número 31. Este registrador é utilizado especificamente para armazenar o endereço de retorno na instrução `JAL`. A seleção da saída é controlada pelo sinal `regdst`, enquanto a saída do multiplexador é representada por `mux_out`.

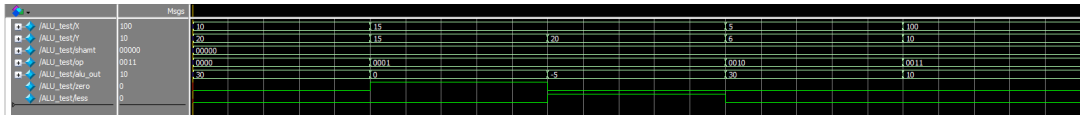
Observa-se que, para cada valor assumido pelo sinal de seleção `regdst`, a saída `mux_out` corresponde corretamente ao registrador especificado. Esse comportamento está de acordo com o mapeamento planejado dos sinais de controle, conforme apresentado na Tabela 8.

5.1.6 Simulando a Unidade Lógica e Aritmética

A Unidade Lógica e Aritmética (ALU) é responsável por realizar operações aritméticas, lógicas e de deslocamento. Para validar seu funcionamento, a simulação foi dividida em três blocos, de acordo com os tipos de operação. As entradas fornecidas à ALU são: `X` e `Y`, correspondentes aos operandos da operação; `shamt`, que define a quantidade de bits a serem deslocados (nas operações de shift); e `op`, que é o código de operação interpretado pela ALU, conforme especificado na Tabela 10. As saídas da ALU são `alu_out`, contendo o resultado da operação, além das flags `zero` e `less`, utilizadas em desvios condicionais.

A Figura 17 apresenta os testes para as operações aritméticas. Para facilitar a análise, os operandos utilizados foram representados em decimal. As operações realizadas, em ordem, foram: adição, duas subtrações (resultando em valores positivos e negativos, respectivamente), multiplicação e divisão. Verifica-se que os resultados e flags produzidos pela ALU estão corretos, de acordo com os valores dos operandos e os códigos de operação fornecidos.

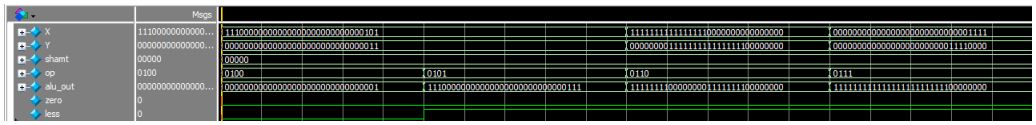
Figura 17 – Simulação da ALU para operações aritméticas



Fonte: O Autor

Na Figura 18, são exibidos os testes das operações lógicas. Foram simuladas, em sequência, as operações `AND`, `OR`, `XOR` e `NOR`, todas com resultados condizentes com os valores de entrada e suas respectivas funções lógicas.

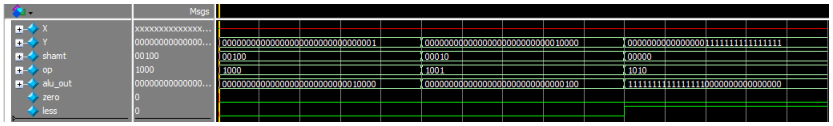
Figura 18 – Simulação da ALU para operações lógicas



Fonte: O Autor

Por fim, a Figura 19 mostra o teste das operações de deslocamento. Nessas instruções, o operando `X` não é utilizado, sendo deixado em branco na simulação. As operações executadas com base em `Y` foram, respectivamente: deslocamento lógico à esquerda (`SLL`), deslocamento lógico à direita (`SRL`) e a instrução `LUI`, que desloca os 16 bits inferiores de `Y` para os 16 bits superiores. Observa-se que os resultados obtidos estão corretos, com os deslocamentos executados conforme o valor de `shamt` ou os 16 bits fixos, no caso da instrução `LUI`.

Figura 19 – Simulação da ALU para operações de deslocamento



Fonte: O Autor

5.1.7 Unidade de Controle da ALU

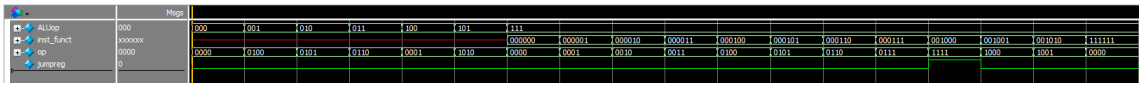
O módulo `ALU_control` é responsável por gerar o código de operação que será fornecido à ALU com base nas entradas recebidas. Para facilitar a visualização do seu

funcionamento, a simulação foi realizada de forma isolada, embora o módulo funcione corretamente quando integrado à ALU.

A entrada `ALUop`, que será proveniente da Unidade de Controle, representa uma codificação preliminar da operação desejada, enquanto o campo `inst_funct` é utilizado para decodificar operações do tipo R. As saídas analisadas são o sinal `op`, que define a operação específica a ser executada pela ALU, e o sinal `jumpreg`, ativado exclusivamente na instrução `JR`.

A Figura 20 apresenta os resultados da simulação. Nos casos em que o campo `inst_funct` está vazio, ele não é necessário para a definição da operação, pois trata-se de instruções que não pertencem ao formato R. A partir do momento em que `ALUop` assume o valor 111, o campo `inst_funct` passa a ser considerado para diferenciar as instruções R. Observa-se que todas as saídas estão de acordo com o comportamento esperado, conforme estabelecido na Tabela 10, o que comprova o funcionamento correto do módulo.

Figura 20 – Simulação do controle da ALU

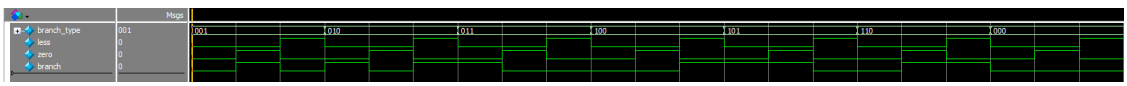


Fonte: O Autor

5.1.8 Simulando a Unidade Lógica de *Branch*

A simulação da Unidade Lógica de *Branch*, apresentada na Figura 21, demonstra o comportamento do módulo responsável por avaliar instruções de desvio condicional. Sua entrada principal é o sinal `branch_type`, que especifica qual tipo de comparação deve ser realizada, conforme definido na Tabela 9. As entradas auxiliares `zero` e `less` são utilizadas para indicar o resultado das comparações entre os operandos da instrução: a flag `zero` é ativada quando os valores são iguais, enquanto `less` indica que o primeiro operando é menor que o segundo. É importante destacar que essas duas flags não são ativadas simultaneamente.

Figura 21 – Simulação da Unidade Lógica de *Branch*



Fonte: O Autor

A saída do módulo, representada pelo sinal `branch`, será 1 quando a condição do desvio for satisfeita e 0 caso contrário. Durante a simulação, foram testadas todas as combinações possíveis para os diferentes valores de `branch_type`. Quando `branch_type` é igual a 001, o desvio deve ocorrer caso os operandos sejam iguais, ou seja, quando `zero` for

1. Já para o valor 010, o desvio ocorre se os operandos forem diferentes, o que se traduz na flag `zero` inativa.

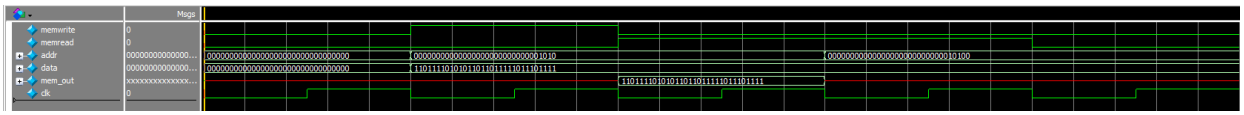
Nos casos em que `branch_type` é 011, o desvio será tomado apenas se o primeiro operando for maior que o segundo, o que implica que a flag `less` deve estar desativada. Para `branch_type` igual a 100, a instrução de desvio ocorre quando o primeiro operando for menor que o segundo, ou seja, quando `less` estiver ativa. Para o valor 101, o desvio é executado se o primeiro operando for maior ou igual ao segundo, o que se confirma com `less` inativo ou `zero` ativo. Já para o tipo 110, o desvio ocorre se o primeiro operando for menor ou igual ao segundo, sendo necessário que ao menos uma das flags, `less` ou `zero`, esteja ativa. Por fim, quando `branch_type` é 000, nenhuma instrução de desvio está sendo processada, e o sinal de `branch` deve permanecer desativado.

Com base nas condições analisadas, observa-se que os resultados apresentados na simulação da Figura 21 estão corretos, validando o funcionamento da Unidade Lógica de Branch conforme o comportamento esperado.

5.1.9 Simulando a Memória de Dados

A simulação da memória de dados considera como entradas os sinais `memwrite` e `memread`, responsáveis por habilitar, respectivamente, a escrita e a leitura na memória. Além disso, há a entrada `addr`, que especifica o endereço de acesso, e `data`, que representa o dado de entrada para escrita. A saída da memória é dada por `mem_out`. Este módulo é síncrono, ou seja, depende de um sinal de *clock* para a execução das operações.

Figura 22 – Simulação da Memória de Dados



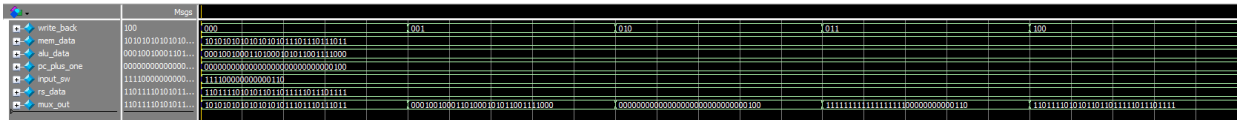
Fonte: O Autor

A simulação, apresentada na Figura 15, mostra que, ao ativar o sinal de escrita com os valores de endereço e dado devidamente fornecidos e, em seguida, desativar o sinal de escrita e ativar o de leitura, observa-se que o dado foi corretamente armazenado e, posteriormente, lido da posição de memória. Adicionalmente, a tentativa de leitura em uma posição não previamente escrita resulta na ausência de valor, o que condiz com o comportamento esperado. Dessa forma, a simulação valida o funcionamento correto da memória de dados.

5.1.10 Simulando a seleção de dados de escrita/saída

A seleção dos dados de escrita/saída é realizada por meio de um multiplexador, que escolhe entre cinco possíveis origens de dados. Na simulação apresentada na Figura 23, o sinal de controle `write_back` determina essa seleção, e as entradas são: `mem_data`, contendo o dado proveniente da memória; `alu_data`, com o resultado da operação da ALU; `pc_plus_one`, representando o valor de PC + 1; `input_sw`, com o valor de entrada fornecido pelos *switches* do kit FPGA; e `rs_data`, que representa o dado lido do banco de registradores no endereço `rs`. A saída do módulo é dada por `mux_out`.

Figura 23 – Simulação da seleção de dados de escrita/saída



Fonte: O Autor

De acordo com a Tabela 9, os valores do sinal `write_back` devem selecionar as entradas conforme: 000 para `mem_data`, 001 para `alu_data`, 010 para `pc_plus_one`, 011 para `input_sw` e 100 para `rs_data`. Dessa forma, observa-se que a simulação confirma o funcionamento correto do módulo.

5.2 Verificação do sistema computacional com algoritmo de lógica de programação

Uma vez que foi-se garantido o comportamento adequado de cada um dos módulos da unidade de processamento, a partir da implementação da unidade de controle e da interface de entrada e saída com o FPGA, a validação do sistema computacional foi visualizada diretamente no kit, verificando o funcionamento do algoritmo proposto na seção de desenvolvimento.

No início da execução, o algoritmo solicita a inserção da quantidade de elementos que serão armazenados na memória. A Figura 24 ilustra esse momento no kit FPGA. Observa-se que o valor binário configurado por meio dos *switches* corresponde ao esperado. Além disso, o LED indicador de instrução de entrada é ativado corretamente, sinalizando o início da operação.

Após a definição da quantidade de entradas, a instrução de salto `JAL` é utilizada para direcionar a execução à rotina de leitura de dados. As cinco inserções realizadas são demonstradas nas Figuras 25, 26, 27, 28 e 29, confirmando o correto armazenamento dos valores na memória.

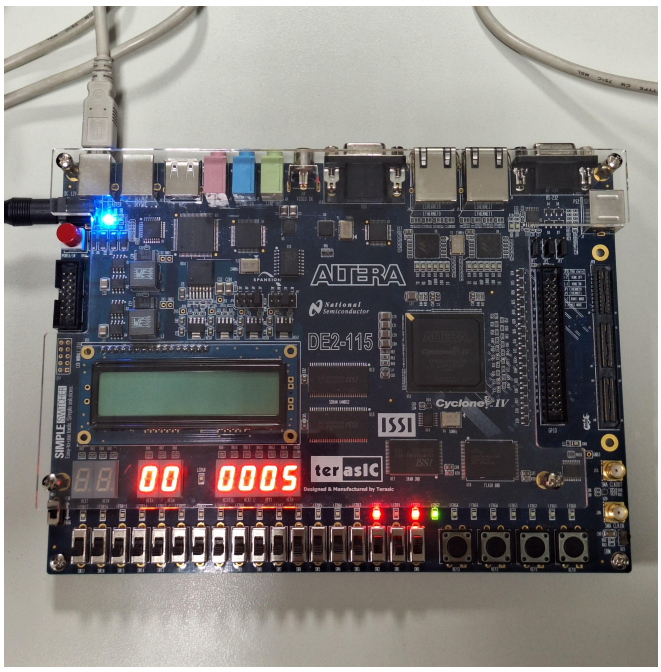
Concluído o processo de entrada, a instrução `JR` realiza o retorno ao ponto de chamada, permitindo a continuidade do programa. A etapa seguinte consiste em um *loop* que percorre sequencialmente os endereços de memória, comparando os valores armazenados. Quando um novo valor máximo ou mínimo é identificado, o registrador correspondente é atualizado. A Figura 30 apresenta esse processo em execução, evidenciando o comportamento esperado do sistema.

Ao final da análise dos dados, o primeiro valor impresso corresponde ao maior elemento encontrado, conforme mostrado na Figura 31. Em seguida, é exibido o menor valor, representado corretamente na Figura 32. Observe que, agora, outro LED verde dedicado a saída foi aceso.

Por fim, a execução alcança a instrução `HALT`, responsável por interromper o funcionamento do processador até que o sinal de `reset` seja ativado. Durante a execução dessa instrução, o primeiro LED verde do kit permanece aceso, conforme ilustrado na Figura 33, validando o comportamento esperado do sistema.

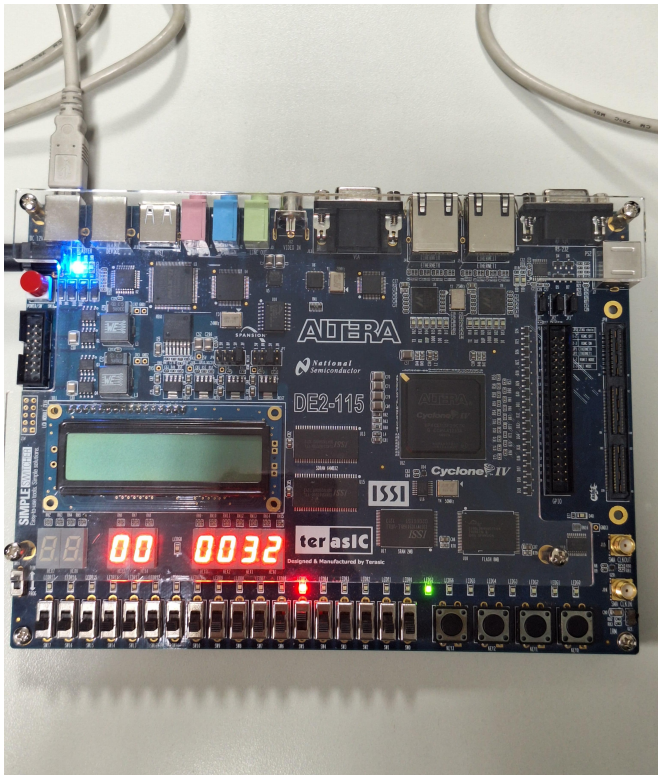
Portanto, foi possível validar o funcionamento do processador implementado, comprovando sua capacidade de executar corretamente um algoritmo representativo de lógica de programação. A interação entre os módulos da unidade de processamento, o controle de fluxo do programa, os acessos à memória e a comunicação com os dispositivos de entrada e saída do kit FPGA ocorreram conforme o projetado. Os resultados obtidos demonstram que a arquitetura desenvolvida é funcional, capaz de interpretar e executar instruções de maneira confiável, além de representar com fidelidade os conceitos teóricos de arquitetura e organização de computadores. Essa validação prática reforça a aplicabilidade do projeto e sua aderência aos objetivos estabelecidos.

Figura 24 – Funcionamento do algoritmo - Entrada do total de inserções



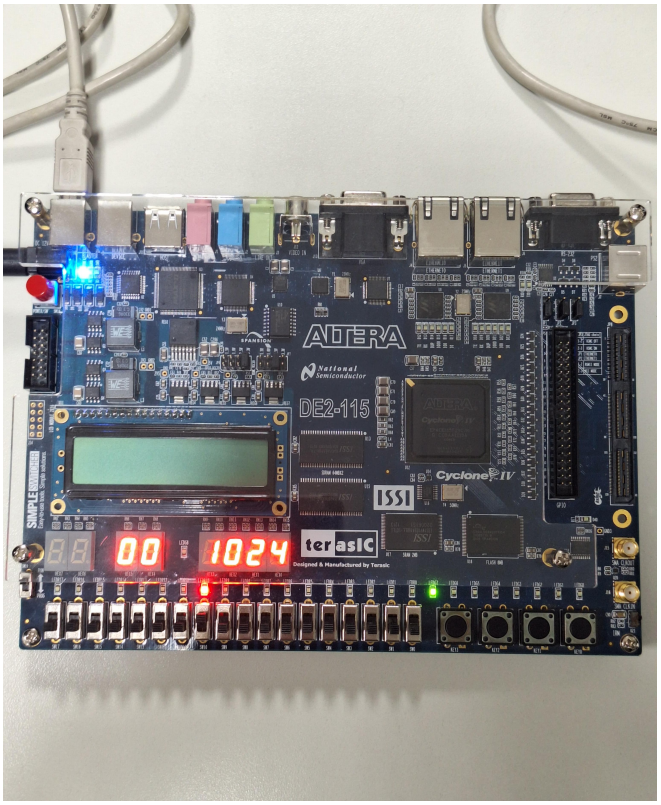
Fonte: O Autor

Figura 25 – Funcionamento do algoritmo - Primeira inserção



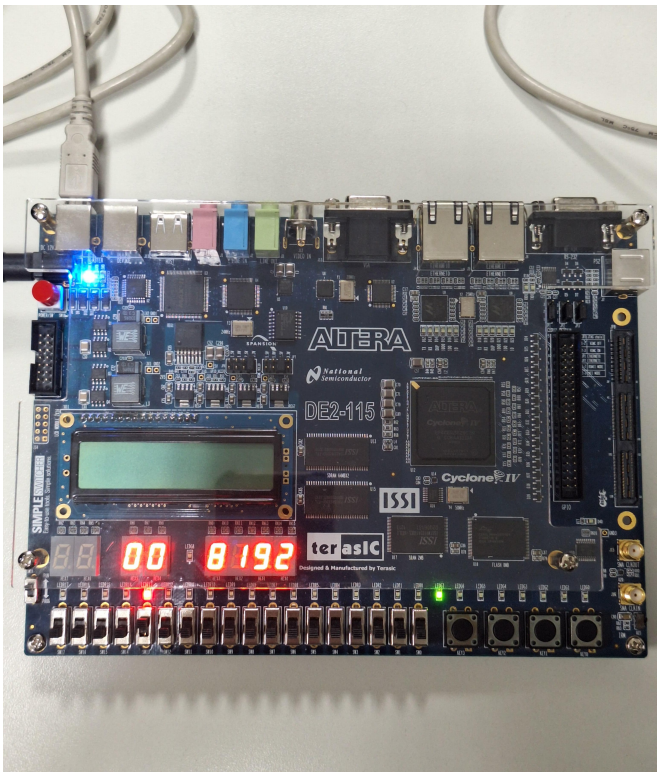
Fonte: O Autor

Figura 26 – Funcionamento do algoritmo - Segunda inserção



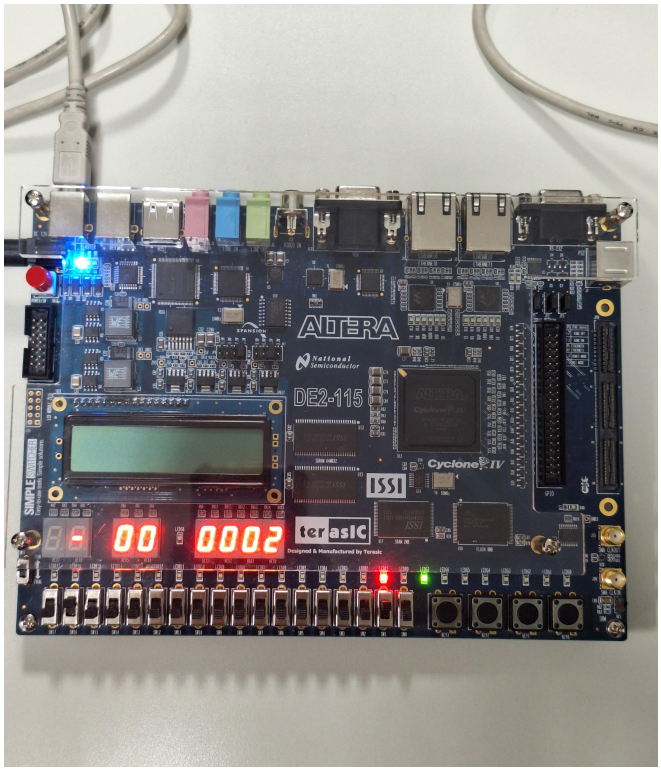
Fonte: O Autor

Figura 27 – Funcionamento do algoritmo - Terceira inserção



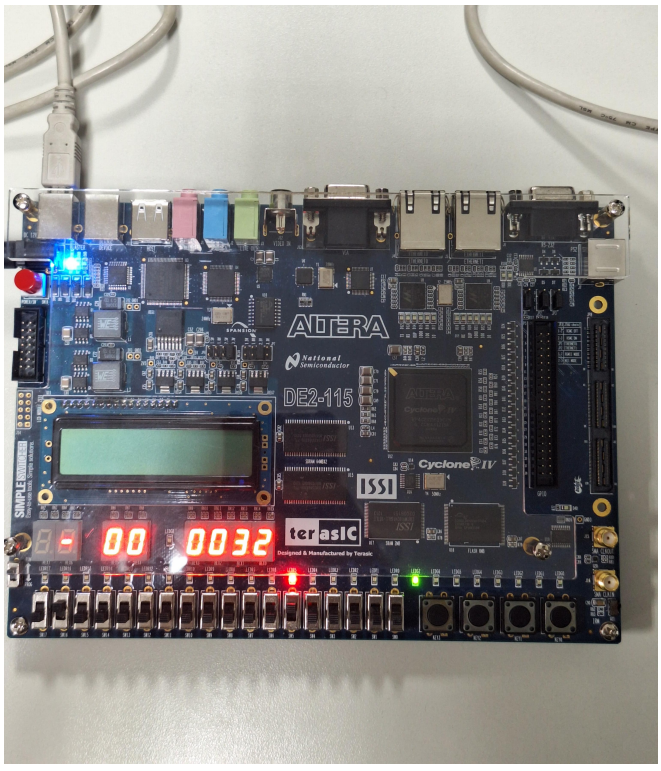
Fonte: O Autor

Figura 28 – Funcionamento do algoritmo - Quarta inserção



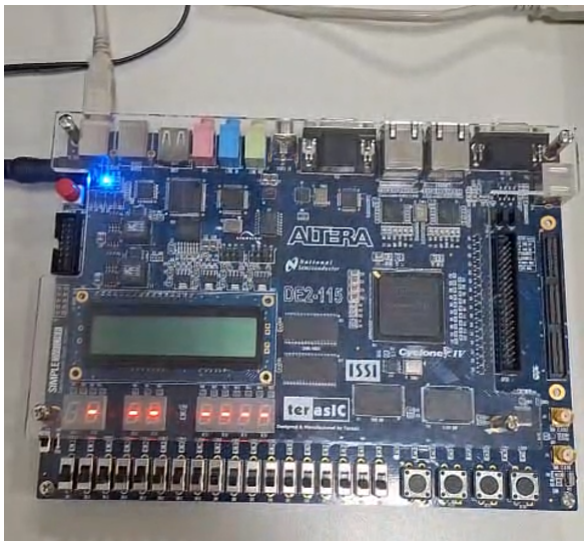
Fonte: O Autor

Figura 29 – Funcionamento do algoritmo - Quinta inserção



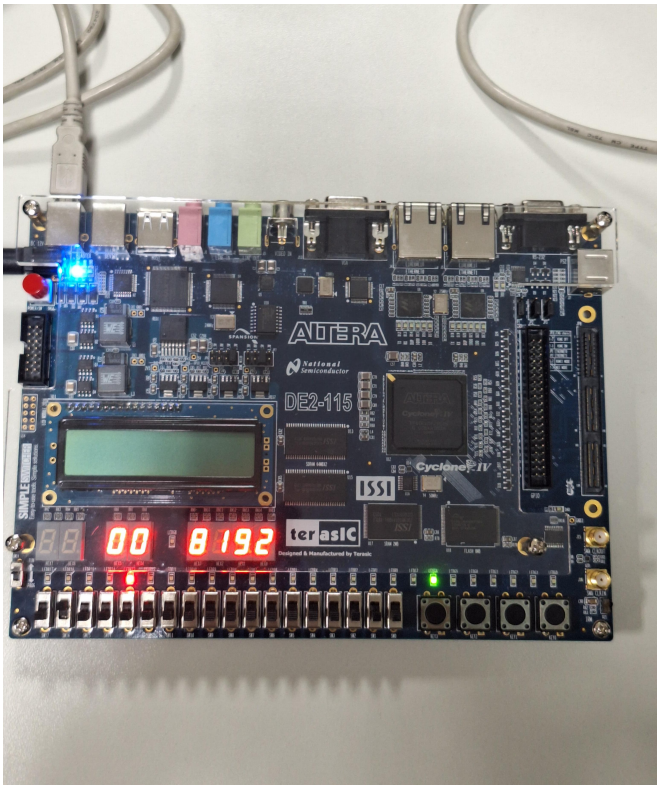
Fonte: O Autor

Figura 30 – Funcionamento do algoritmo - *Loop* de execução



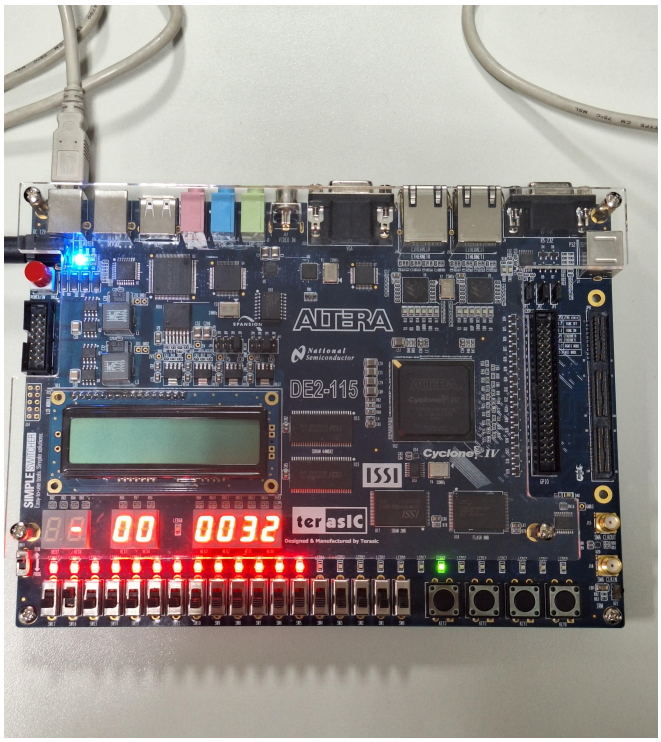
Fonte: O Autor

Figura 31 – Funcionamento do algoritmo - Impressão do maior valor



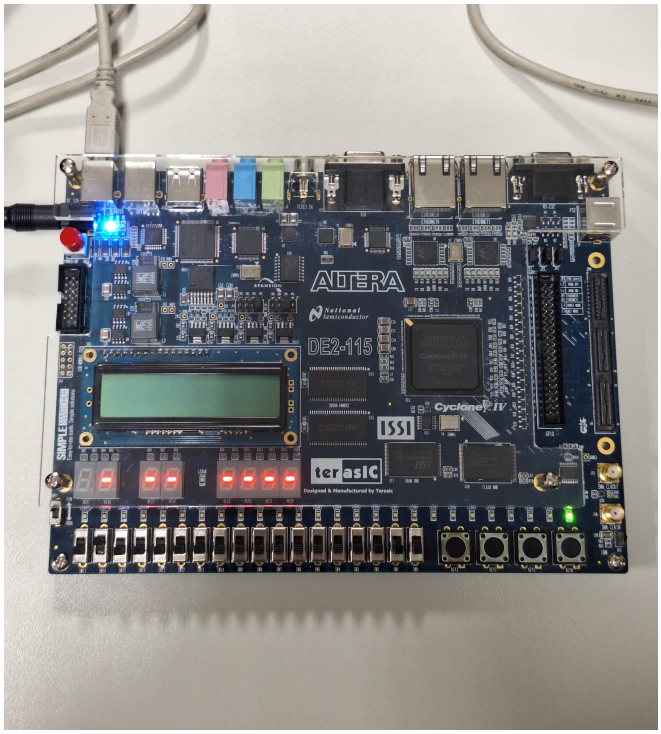
Fonte: O Autor

Figura 32 – Funcionamento do algoritmo - Impressão do menor valor



Fonte: O Autor

Figura 33 – Funcionamento do algoritmo - Instrução de Halt



Fonte: O Autor

6 Considerações Finais

O presente trabalho teve como objetivo o desenvolvimento completo de um processador baseado na arquitetura MIPS 32, implementado em um kit FPGA e capaz de executar algoritmos básicos de lógica de programação. A partir do projeto arquitetural definido, que contemplou o conjunto de instruções, o caminho de dados e os sinais de controle, foi possível implementar e validar um sistema computacional funcional, com interação direta por meio dos dispositivos físicos disponíveis no kit.

A validação realizada por meio da execução de um algoritmo de lógica de programação demonstrou que o processador desenvolvido opera corretamente, sendo capaz de realizar operações de entrada, armazenamento, comparação e saída de dados, o que atesta sua capacidade de interpretar e executar instruções alinhadas aos objetivos propostos.

Durante o processo de desenvolvimento, a principal dificuldade enfrentada esteve relacionada à implementação do comportamento de parada do processador para a interação de entrada e saída do usuário. A solução adotada — embora funcional — não refletiu, inicialmente, a abordagem ideal prevista, representando um ponto que pode ser aprimorado em versões futuras do projeto.

Além disso, outro aspecto que se apresenta como possível melhoria está na utilização do *display* LCD do kit FPGA, o qual poderia oferecer uma interface de saída mais interessante e completa para o usuário.

Por fim, o trabalho proporcionou uma aplicação prática sólida dos conhecimentos adquiridos em arquitetura e organização de computadores, consolidando a capacidade de projetar, implementar e validar um processador a partir de especificações arquiteturais bem definidas.

Referências

1 STALLINGS, W. *Arquitetura e organização de computadores*. 8.ed. ed. São Paulo: Pearson Education, 2010. Citado 6 vezes nas páginas 8, 11, 12, 13, 15 e 17.

2 WEBER, R. F. *Fundamentos de arquitetura de computadores*. 3.ed.. ed. Porto Alegre, RS: Bookman, 2008. Citado 5 vezes nas páginas 10, 17, 18, 19 e 20.

3 COSTA, C. da. *Projetos de circuitos digitais com FPGA*. 3.ed. ed. São Paulo: Érica, 2014. Citado na página 10.

4 TERASIC. *DE2-115: User Manual*. [S.l.]: Terasic Technologies, 2017. 121 p. Citado na página 11.

5 WANHAMMAR, L. 8 - dsp architectures. In: WANHAMMAR, L. (Ed.). *DSP Integrated Circuits*. Burlington: Academic Press, 1999, (Academic Press Series in Engineering). p. 357–385. ISBN 978-0-12-734530-7. Disponível em: <https://www.sciencedirect.com/science/article/pii/B9780127345307500088>. Citado na página 12.

6 PATTERSON, D. A.; HENNESY, J. L. *Organização e projeto de computadores*. 5.ed. ed. Rio de Janeiro: Elsevier, 2017. Citado 4 vezes nas páginas 12, 14, 15 e 16.

7 PATTERSON, D. A.; HENNESY, J. L. *Organização e projeto de computadores*. 3.ed. ed. Rio de Janeiro: Campus, 2005. Citado 5 vezes nas páginas 14, 21, 22, 23 e 26.

8 TOCCI, R. J.; WIDMER, N. S.; MOSS, G. L. *Sistemas Digitais: princípios e aplicações*. 11.ed. ed. São Paulo: Pearson, 2011. Citado 3 vezes nas páginas 16, 18 e 19.

9 FPGA4Students. *Verilog Code for Clock Divider on FPGA*. <https://www.fpga4student.com/2017/08/verilog-code-for-clock-divider-on-fpga.html>. Acessado em 10 de julho de 2025. Citado na página 38.

10 FPGA4Students. *Verilog code for debouncing buttons on FPGA*. <https://www.fpga4student.com/2017/04/simple-debouncing-verilog-code-for.html>. Acessado em 10 de julho de 2025. Citado na página 38.

Apêndices

APÊNDICE A – system_top.sv

Código do arquivo system_top.sv

```
1  module system_top(
2      input logic clk, read_clk,
3      input logic reset, btn_io,
4      input logic [15:0] switches,
5      input logic sw_negative,
6      output logic input_led, output_led, halt_led,
7      output logic [6:0] HEX6, HEX5, HEX4, HEX3, HEX2, HEX1, HEX0,
8      output logic [17:0] data_led
9  );
10     logic [31:0] data_out;
11     logic [19:0] bin_value_mux;
12     logic flag_in, flag_out, flag_negative;
13     logic btn_debounced;
14     logic flag_io_negative;
15     logic clk_processor;
16
17     assign bin_value_mux = flag_in ? {4'b0, switches} : data_out
           [19:0]; //unsigned input
18     assign flag_io_negative = flag_in ? sw_negative :
           flag_negative;
19
20     assign data_led = bin_value_mux;
21
22     clock_divider clk_divider(
23         .clk_in(clk),
24         .clk_out(clk_processor)
25     );
26
27     button_debouncer btn_debouncer(
28         .btn_1(btn_io),
29         .clk(clk),
30         .btn_out(btn_debounced)
31     );
32
33     mips_processor mips(
34         .clk(clk_processor),
```

```
35          .read_clk(read_clk),
36          .btn_io(btn_debounced),
37          .reset(reset),
38          .switches(switches),
39          .sw_negative(sw_negative),
40          .data_out(data_out),
41          .flag_in(flag_in),
42          .flag_out(flag_out),
43          .flag_negative(flag_negative),
44          .flag_halt(halt_led)
45      );
46
47      display_output display(
48          .flag_out(flag_out),
49          .flag_in(flag_in),
50          .flag_negative(flag_io_negative),
51          .bin_value(bin_value_mux),
52          .HEX6(HEX6),
53          .HEX5(HEX5),
54          .HEX4(HEX4),
55          .HEX3(HEX3),
56          .HEX2(HEX2),
57          .HEX1(HEX1),
58          .HEX0(HEX0)
59      );
60
61      assign input_led = flag_in;
62      assign output_led = flag_out;
63
64  endmodule
```

APÊNDICE B – mips_processor.sv

Código do arquivo mips_processor.sv

```
1  module mips_processor(
2      input logic clk, read_clk,
3      input logic reset, btn_io,
4      input logic [15:0] switches,
5      input logic sw_negative,
6      output logic [31:0] data_out,
7      output logic flag_in, flag_out, flag_negative, flag_halt
8  );
9
10     logic [31:0] pc_addr;
11     logic [31:0] pc_next_addr;
12     logic [31:0] reg2_data;
13     logic [31:0] inst;
14     logic [31:0] inst_imm_extended, alu_Y , mem_data, reg1_data,
          alu_data;
15     logic [31:0] input_sw, input_data;
16     logic [4:0] write_reg_addr;
17     logic [3:0] flag_alu_operation;
18     logic [2:0] flag_branch_type, flag_alu_op, flag_write_back;
19     logic [1:0] flag_regdst;
20     logic flag_regwrite, flag_jump, flag_alu_src, flag_memwrite,
          flag_memread;
21     logic flag_jumpreg, flag_branch, flag_io_stall;
22     logic flag_zero, flag_less;
23
24     assign input_sw = {16'b0, switches};
25     assign input_data = sw_negative ? (~input_sw + 32'd1) :
          input_sw;
26
27     pc_update_logic pc_update_logic(
28         .pc_current(pc_addr),
29         .reg1_data(reg1_data),
30         .inst_target(inst[25:0]),
31         .inst_offset(inst_imm_extended),
32         .jumpreg(flag_jumpreg),
33         .branch(flag_branch),
```

```
34          .jump(flag_jump),
35          .pc_next(pc_next_addr)
36      );
37
38      program_counter program_counter(
39          .clk(clk),
40          .reset(reset),
41          .halt(flag_halt),
42          .clk_manual(btn_io),
43          .io_stall(flag_io_stall),
44          .pc_in(pc_next_addr),
45          .pc_out(pc_addr)
46      );
47
48      instruction_ROM instruction_memory(
49          .clk(read_clk),
50          .addr(pc_addr[7:0]),
51          .inst_code(inst)
52      );
53
54      control_unit control_unit(
55          .inst_op(inst[31:26]),
56          .writeback(flag_write_back),
57          .branch_type(flag_branch_type),
58          .aluop(flag_alu_op),
59          .regdst(flag_regdst),
60          .jump(flag_jump),
61          .memread(flag_memread),
62          .memwrite(flag_memwrite),
63          .alusrc(flag_alu_src),
64          .regwrite(flag_regwrite),
65          .out(flag_out),
66          .in(flag_in),
67          .halt(flag_halt),
68          .io_stall(flag_io_stall)
69      );
70
71      mux_write_reg mux_write_reg(
72          .regdst(flag_regdst),
73          .inst_rt(inst[20:16]),
74          .inst_rd(inst[15:11]),
75          .reg31_addr(5'b11111),
```

```
76          .mux_out(write_reg_addr)
77      );
78
79      register_file register_file(
80          .reg1_addr(inst[25:21]),
81          .reg2_addr(inst[20:16]),
82          .write_reg_addr(write_reg_addr),
83          .write_data(data_out),
84          .reg1_data(reg1_data),
85          .reg2_data(reg2_data),
86          .clk(clk),
87          .regwrite(flag_regwrite)
88      );
89
90      sign_extension sign_extension_imm(
91          .inst_imm(inst[15:0]),
92          .inst_imm_extended(inst_imm_extended)
93      );
94
95      mux_2to1 mux_alusrc(
96          .data1(reg2_data),
97          .data2(inst_imm_extended),
98          .flag(flag_alu_src),
99          .mux_out(alu_Y)
100     );
101
102     ALU_control ALU_control(
103         .inst_funct(inst[5:0]),
104         .ALUop(flag_alu_op),
105         .ALUoperation(flag_alu_operation),
106         .jumpreg(flag_jumpreg)
107     );
108
109     ALU ALU(
110         .X(reg1_data),
111         .Y(alu_Y),
112         .shamt(inst[10:6]),
113         .op(flag_alu_operation),
114         .alu_out(alu_data),
115         .zero(flag_zero),
116         .less(flag_less)
117     );
```

```
118
119     branch_logic_unit branch_logic_unit(
120         .branch_type(flag_branch_type),
121         .alu_less(flag_less),
122         .alu_zero(flag_zero),
123         .branch(flag_branch)
124     );
125
126     data_RAM data_memory(
127         .read_addr(alu_data[9:0]),
128         .write_addr(alu_data[9:0]),
129         .data(reg2_data),
130         .read_clk(read_clk),
131         .write_clk(clk),
132         .memwrite(flag_memwrite),
133         .memread(flag_memread),
134         .mem_out(mem_data)
135     );
136
137     mux_write_back mux_write_back(
138         .write_back(flag_write_back),
139         .mem_data(mem_data),
140         .alu_data(alu_data),
141         .pc_plus_one(pc_addr + 1),
142         .input_data(input_data),
143         .rs_data(reg1_data),
144         .mux_out(data_out)
145     );
146
147     assign flag_negative = data_out[31];
148
149 endmodule
```

APÊNDICE C – program_counter.sv

Código do arquivo program_counter.sv

```
1  module program_counter(
2          input logic clk,
3          input logic halt,
4          input logic reset,
5          input logic clk_manual,
6          input logic io_stall,
7          input logic [31:0] pc_in,
8          output logic [31:0] pc_out
9  );
10         logic clk_processor;
11
12         assign clk_processor = io_stall ? clk_manual : clk;
13
14         always_ff @(posedge clk_processor or posedge reset) begin
15                 if (reset)
16                         pc_out <= 32'b0;
17                 else if (~halt)
18                         pc_out <= pc_in;
19         end
20
21 endmodule
```

APÊNDICE D – pc_update_logic.sv

Código do arquivo pc_update_logic.sv

```
1  module pc_update_logic(
2          input logic [31:0] pc_current,
3          input logic [31:0] reg1_data,
4          input logic [25:0] inst_target,
5          input logic [31:0] inst_offset,
6          input logic jumpreg,
7          input logic branch,
8          input logic jump,
9          output logic [31:0] pc_next
10 );
11
12         logic [31:0] pc_plus_1;
13         logic [31:0] pc_branch;
14         logic [31:0] pc_jr_or_pc1;
15         logic [31:0] pc_branch_or_jr;
16         logic [31:0] pc_jump_addr;
17
18         assign pc_plus_1 = pc_current + 1;
19         assign pc_branch = pc_current + 1 + inst_offset;
20         assign pc_jump_addr = {pc_current[31:26], inst_target};
21
22         // MUX para JR
23         mux_2to1 mux_jr (
24                 .flag(jumpreg),
25                 .data1(pc_plus_1),
26                 .data2(reg1_data),
27                 .mux_out(pc_jr_or_pc1)
28         );
29
30         // MUX para BRANCH
31         mux_2to1 mux_br (
32                 .flag(branch),
33                 .data1(pc_jr_or_pc1),
34                 .data2(pc_branch),
35                 .mux_out(pc_branch_or_jr)
36         );
```

```
37
38          // MUX para JUMP
39          mux_2to1 mux_j (
40                  .flag(jump),
41                  .data1(pc_branch_or_jr),
42                  .data2(pc_jump_addr),
43                  .mux_out(pc_next)
44          );
45
46  endmodule
```

APÊNDICE E – instruction_ROM.sv

Código do arquivo instruction_ROM.sv

```
1  module instruction_ROM #(
2          parameter init_file = "rom_init.mif",
3          parameter ADDR_WIDTH=8
4  ) (
5          input logic clk,
6          input logic [ADDR_WIDTH - 1:0] addr,
7          output logic [31:0] inst_code
8  );
9
10         logic [31:0] rom[2**ADDR_WIDTH-1:0];
11
12         initial begin
13                 $readmemb(init_file, rom);
14         end
15
16         always_ff @ (posedge clk) begin
17                 inst_code = rom[addr];
18         end
19
20 endmodule
```

APÊNDICE F – control_unit.sv

Código do arquivo control_unit.sv

```
1  module control_unit(
2      input logic [5:0] inst_op,
3      output logic [2:0] writeback,
4      output logic [2:0] branch_type,
5      output logic [2:0] aluop,
6      output logic [1:0] regdst,
7      output logic jump,
8      output logic memread,
9      output logic memwrite,
10     output logic alusrc,
11     output logic regwrite,
12     output logic out,
13     output logic in,
14     output logic halt,
15     output logic io_stall
16 );
17
18     parameter    R_type = 6'b000000,
19                  ADDI = 6'b000001,
20                  ANDI = 6'b000010,
21                  ORI = 6'b000011,
22                  XORI = 6'b000100,
23                  LW = 6'b000101,
24                  SW = 6'b000110,
25                  LUI = 6'b000111,
26                  BEQ = 6'b001000,
27                  BNE = 6'b001001,
28                  BLT = 6'b001010,
29                  BGT = 6'b001011,
30                  BLE = 6'b001100,
31                  BGE = 6'b001101,
32                  IN = 6'b001110,
33                  OUT = 6'b001111,
34                  J = 6'b010000,
35                  JAL = 6'b010001,
36                  NOP = 6'b010010,
```

```
37                HALT = 6'b010011;
38
39
40      always_comb begin
41          writeback   = 3'b000;
42          branch_type = 3'b000;
43          aluop       = 3'b000;
44          regdst      = 2'b00;
45          jump        = 0;
46          memread     = 0;
47          memwrite    = 0;
48          alusrc      = 0;
49          regwrite    = 0;
50          out         = 0;
51          in          = 0;
52          halt        = 0;
53          io_stall    = 0;
54
55          case(inst_op)
56              R_type: begin
57                  writeback = 3'b001;
58                  branch_type = 3'b000;
59                  aluop = 3'b111;
60                  regdst = 2'b10;
61                  jump = 0;
62                  memread = 0;
63                  memwrite = 0;
64                  alusrc = 0;
65                  regwrite = 1;
66                  out = 0;
67                  in = 0;
68                  halt = 0;
69                  io_stall = 0;
70              end
71
72              ADDI: begin
73                  writeback = 3'b001;
74                  branch_type = 3'b000;
75                  aluop = 3'b000;
76                  regdst = 2'b01;
77                  jump = 0;
78                  memread = 0;
```

```
79                  memwrite = 0;
80                  alusrc = 1;
81                  regwrite = 1;
82                  out = 0;
83                  in = 0;
84                  halt = 0;
85                  io_stall = 0;
86              end
87
88              ANDI: begin
89                  writeback = 3'b001;
90                  branch_type = 3'b000;
91                  aluop = 3'b001;
92                  regdst = 2'b01;
93                  jump = 0;
94                  memread = 0;
95                  memwrite = 0;
96                  alusrc = 1;
97                  regwrite = 1;
98                  out = 0;
99                  in = 0;
100                 halt = 0;
101                 io_stall = 0;
102             end
103
104             ORI: begin
105                 writeback = 3'b001;
106                 branch_type = 3'b000;
107                 aluop = 3'b010;
108                 regdst = 2'b01;
109                 jump = 0;
110                 memread = 0;
111                 memwrite = 0;
112                 alusrc = 1;
113                 regwrite = 1;
114                 out = 0;
115                 in = 0;
116                 halt = 0;
117                 io_stall = 0;
118             end
119
120             XORI: begin
```

```
121                 writeback = 3'b001;
122                 branch_type = 3'b000;
123                 aluop = 3'b011;
124                 regdst = 2'b01;
125                 jump = 0;
126                 memread = 0;
127                 memwrite = 0;
128                 alusrc = 1;
129                 regwrite = 1;
130                 out = 0;
131                 in = 0;
132                 halt = 0;
133                 io_stall = 0;
134             end
135
136             LW: begin
137                 writeback = 3'b000;
138                 branch_type = 3'b000;
139                 aluop = 3'b000;
140                 regdst = 2'b01;
141                 jump = 0;
142                 memread = 1;
143                 memwrite = 0;
144                 alusrc = 1;
145                 regwrite = 1;
146                 out = 0;
147                 in = 0;
148                 halt = 0;
149                 io_stall = 0;
150             end
151
152             SW: begin
153                 writeback = 3'b000;
154                 branch_type = 3'b000;
155                 aluop = 3'b000;
156                 regdst = 2'b00;
157                 jump = 0;
158                 memread = 0;
159                 memwrite = 1;
160                 alusrc = 1;
161                 regwrite = 0;
162                 out = 0;
```

```
163                 in = 0;
164                 halt = 0;
165                 io_stall = 0;
166             end
167
168             LUI: begin
169                 writeback = 3'b001;
170                 branch_type = 3'b000;
171                 aluop = 3'b101;
172                 regdst = 2'b01;
173                 jump = 0;
174                 memread = 0;
175                 memwrite = 0;
176                 alusrc = 1;
177                 regwrite = 1;
178                 out = 0;
179                 in = 0;
180                 halt = 0;
181                 io_stall = 0;
182             end
183
184             BEQ: begin
185                 writeback = 3'b000;
186                 branch_type = 3'b001;
187                 aluop = 3'b100;
188                 regdst = 2'b00;
189                 jump = 0;
190                 memread = 0;
191                 memwrite = 0;
192                 alusrc = 0;
193                 regwrite = 0;
194                 out = 0;
195                 in = 0;
196                 halt = 0;
197                 io_stall = 0;
198             end
199
200             BNE: begin
201                 writeback = 3'b000;
202                 branch_type = 3'b010;
203                 aluop = 3'b100;
204                 regdst = 2'b00;
```

```
205                 jump = 0;
206                 memread = 0;
207                 memwrite = 0;
208                 alusrc = 0;
209                 regwrite = 0;
210                 out = 0;
211                 in = 0;
212                 halt = 0;
213                 io_stall = 0;
214             end
215
216             BLT: begin
217                 writeback = 3'b000;
218                 branch_type = 3'b011;
219                 aluop = 3'b100;
220                 regdst = 2'b00;
221                 jump = 0;
222                 memread = 0;
223                 memwrite = 0;
224                 alusrc = 0;
225                 regwrite = 0;
226                 out = 0;
227                 in = 0;
228                 halt = 0;
229                 io_stall = 0;
230             end
231
232             BGT: begin
233                 writeback = 3'b000;
234                 branch_type = 3'b100;
235                 aluop = 3'b100;
236                 regdst = 2'b00;
237                 jump = 0;
238                 memread = 0;
239                 memwrite = 0;
240                 alusrc = 0;
241                 regwrite = 0;
242                 out = 0;
243                 in = 0;
244                 halt = 0;
245                 io_stall = 0;
246             end
```

```
247
248             BLE: begin
249                 writeback = 3'b000;
250                 branch_type = 3'b101;
251                 aluop = 3'b100;
252                 regdst = 2'b00;
253                 jump = 0;
254                 memread = 0;
255                 memwrite = 0;
256                 alusrc = 0;
257                 regwrite = 0;
258                 out = 0;
259                 in = 0;
260                 halt = 0;
261                 io_stall = 0;
262             end
263
264             BGE: begin
265                 writeback = 3'b000;
266                 branch_type = 3'b110;
267                 aluop = 3'b100;
268                 regdst = 2'b00;
269                 jump = 0;
270                 memread = 0;
271                 memwrite = 0;
272                 alusrc = 0;
273                 regwrite = 0;
274                 out = 0;
275                 in = 0;
276                 halt = 0;
277                 io_stall = 0;
278             end
279
280             IN: begin
281                 writeback = 3'b011;
282                 branch_type = 3'b000;
283                 aluop = 3'b000;
284                 regdst = 2'b01;
285                 jump = 0;
286                 memread = 0;
287                 memwrite = 0;
288                 alusrc = 0;
```

```
289                 regwrite = 1;
290                 out = 0;
291                 in = 1;
292                 halt = 0;
293                 io_stall = 1;
294             end
295
296             OUT: begin
297                 writeback = 3'b100;
298                 branch_type = 3'b000;
299                 aluop = 3'b000;
300                 regdst = 2'b00;
301                 jump = 0;
302                 memread = 0;
303                 memwrite = 0;
304                 alusrc = 0;
305                 regwrite = 0;
306                 out = 1;
307                 in = 0;
308                 halt = 0;
309                 io_stall = 1;
310             end
311
312             J: begin
313                 writeback = 3'b000;
314                 branch_type = 3'b000;
315                 aluop = 3'b000;
316                 regdst = 2'b00;
317                 jump = 1;
318                 memread = 0;
319                 memwrite = 0;
320                 alusrc = 0;
321                 regwrite = 0;
322                 out = 0;
323                 in = 0;
324                 halt = 0;
325                 io_stall = 0;
326             end
327
328             JAL: begin
329                 writeback = 3'b010;
330                 branch_type = 3'b000;
```

```
331                 aluop = 3'b000;
332                 regdst = 2'b11;
333                 jump = 1;
334                 memread = 0;
335                 memwrite = 0;
336                 alusrc = 0;
337                 regwrite = 1;
338                 out = 0;
339                 in = 0;
340                 halt = 0;
341                 io_stall = 0;
342             end
343
344             NOP: begin
345                 writeback = 3'b111;
346                 branch_type = 3'b000;
347                 aluop = 3'b000;
348                 regdst = 2'b00;
349                 jump = 0;
350                 memread = 0;
351                 memwrite = 0;
352                 alusrc = 0;
353                 regwrite = 0;
354                 out = 0;
355                 in = 0;
356                 halt = 0;
357                 io_stall = 0;
358             end
359
360             HALT: begin
361                 writeback = 3'b111;
362                 branch_type = 3'b000;
363                 aluop = 3'b000;
364                 regdst = 2'b00;
365                 jump = 0;
366                 memread = 0;
367                 memwrite = 0;
368                 alusrc = 0;
369                 regwrite = 0;
370                 out = 0;
371                 in = 0;
372                 halt = 1;
```

```
373                 io_stall = 0;
374             end
375             default: begin
376                 writeback   = 3'b000;
377                 branch_type = 3'b000;
378                 aluop       = 3'b000;
379                 regdst      = 2'b00;
380                 jump        = 0;
381                 memread     = 0;
382                 memwrite    = 0;
383                 alusrc      = 0;
384                 regwrite    = 0;
385                 out         = 0;
386                 in          = 0;
387                 halt        = 0;
388                 io_stall    = 0;
389             end
390         endcase
391     end
392
393 endmodule
```

APÊNDICE G – sign_extension.sv

Código do arquivo sign_extension.sv

```
1 module sign_extension(
2         input logic [15:0] inst_imm,
3         output logic [31:0] inst_imm_extended
4 );
5         assign inst_imm_extended = {{16{inst_imm[15]}}, inst_imm
             };
6 endmodule
```

APÊNDICE H – register_file.sv

Código do arquivo register_file.sv

```
1  module register_file(
2          input logic clk,
3          input logic regwrite,
4          input logic[4:0] reg1_addr,
5          input logic [4:0] reg2_addr,
6          input logic [4:0] write_reg_addr,
7          input logic [31:0] write_data,
8
9          output logic [31:0] reg2_data,
10         output logic [31:0] reg1_data
11 );
12
13         logic [31:0] registers [31:0];
14
15         // Escrita s ncrona
16         always_ff @(posedge clk) begin
17                 if (regwrite && write_reg_addr != 0) begin
18                         registers[write_reg_addr] <= write_data;
19                 end
20         end
21
22         // Leitura combinacional
23         assign reg1_data = (reg1_addr != 0) ? registers[reg1_addr
              ] : 32'b0;
24         assign reg2_data = (reg2_addr != 0) ? registers[reg2_addr
              ] : 32'b0;
25
26 endmodule
```

APÊNDICE I – mux_write_reg.sv

Código do arquivo mux_write_reg.sv

```
1  module mux_write_reg (
2          input logic [1:0] regdst,
3          input logic [4:0] inst_rt,
4          input logic [4:0] inst_rd,
5          input logic [4:0] reg31_addr,
6          output logic [4:0] mux_out
7  );
8
9          always_comb begin
10                 case (regdst)
11                         2'b01: mux_out = inst_rt;
12                         2'b10: mux_out = inst_rd;
13                         2'b11: mux_out = reg31_addr;
14                         default: mux_out = 5'b00000;
15                 endcase
16         end
17
18 endmodule
```

APÊNDICE J – ALU.sv

Código do arquivo ALU.sv

```
1  module ALU(
2          input logic [31:0] X, Y,
3          input logic [4:0] shamt,
4          input logic [3:0] op,
5          output logic [31:0] alu_out,
6          output logic zero,
7          output logic less
8  );
9
10         parameter       ADD = 4'b0000,
11                                 SUB = 4'b0001,
12                                 MUL = 4'b0010,
13                                 DIV = 4'b0011,
14                                 AND = 4'b0100,
15                                 OR = 4'b0101,
16                                 XOR = 4'b0110,
17                                 NOR = 4'b0111,
18                                 SLL = 4'b1000,
19                                 SRL = 4'b1001,
20                                 LUI = 4'b1010;
21         always_comb begin
22                 case(op)
23                         ADD: alu_out = X + Y;
24                         SUB: alu_out = X - Y;
25                         MUL: alu_out = X * Y;
26                         DIV: alu_out = X / Y;
27                         AND: alu_out = X & Y;
28                         OR: alu_out = X | Y;
29                         XOR: alu_out = X ^ Y;
30                         NOR: alu_out = ~(X | Y);
31                         SLL: alu_out = Y << shamt;
32                         SRL: alu_out = Y >> shamt;
33                         LUI: alu_out = Y << 16;
34                         default: alu_out = 32'b0;
35                 endcase
36
```

```
37          end
38
39          assign less = alu_out[31];
40          assign zero = (alu_out == 0);
41
42  endmodule
```

APÊNDICE K – mux_2to1.sv

Código do arquivo mux_2to1.sv

```
1  module mux_2to1(
2          input logic [31:0] data1,
3          input logic [31:0] data2,
4          input logic flag,
5          output logic [31:0] mux_out
6  );
7
8          always_comb begin
9                  case (flag)
10                         1'b0: mux_out = data1;
11                         2'b1: mux_out = data2;
12                 endcase
13         end
14
15 endmodule
```

APÊNDICE L – ALU_control.sv

Código do arquivo ALU_control.sv

```
1  module ALU_control(
2      input logic [5:0] inst_funct,
3      input logic [2:0] ALUop,
4      output logic [3:0] ALUoperation,
5
6      output logic jumpreg
7  );
8
9      always_comb begin
10         // default
11         jumpreg = 1'b0;
12         ALUoperation = 4'b0000;
13
14         unique case (ALUop)
15             3'b000: ALUoperation = 4'b0000; // Soma
16             3'b001: ALUoperation = 4'b0100; // AND
17             3'b010: ALUoperation = 4'b0101; // OR
18             3'b011: ALUoperation = 4'b0110; // XOR
19             3'b100: ALUoperation = 4'b0001; // Subtra  o
20             3'b101: ALUoperation = 4'b1010; // LUI (<< 16)
21             3'b111: begin
22                 unique case (inst_funct)
23                     6'b000000: ALUoperation = 4'b0000; // Soma
24                     6'b000001: ALUoperation = 4'b0001; //
                           Subtra  o
25                     6'b000010: ALUoperation = 4'b0010; //
                           Multiplica  o
26                     6'b000011: ALUoperation = 4'b0011; //
                           Divis o
27                     6'b000100: ALUoperation = 4'b0100; // AND
28                     6'b000101: ALUoperation = 4'b0101; // OR
29                     6'b000110: ALUoperation = 4'b0110; // XOR
30                     6'b000111: ALUoperation = 4'b0111; // NOR
31                     6'b001000: begin
32                         ALUoperation = 4'b1111; // Jump Register
33                         jumpreg = 1'b1;
```

```
34                      end
35                      6'b001001: ALUoperation = 4'b1000; // SLL
36                      6'b001010: ALUoperation = 4'b1001; // SRL
37                      default:   ALUoperation = 4'b0000;
38                  endcase
39              end
40              default: ALUoperation = 4'b0000;
41          endcase
42      end
43
44  endmodule
```

APÊNDICE M – branch_logic_unit.sv

Código do arquivo branch_logic_unit.sv

```
1  module branch_logic_unit(
2          input logic alu_less,
3          input logic alu_zero,
4          input logic [2:0] branch_type,
5          output logic branch
6  );
7          parameter       BEQ = 3'b001,
8                                  BNE = 3'b010,
9                                  BLT = 3'b011,
10                                 BGT = 3'b100,
11                                 BLE = 3'b101,
12                                 BGE = 3'b110;
13
14         always_comb begin
15                 case (branch_type)
16                         BEQ: branch = alu_zero;
17                         BNE: branch = ~alu_zero;
18                         BLT: branch = alu_less;
19                         BGT: branch = ~alu_less & ~alu_zero;
20                         BLE: branch = alu_less | alu_zero;
21                         BGE: branch = ~alu_less | alu_zero;
22                         default: branch = 1'b0;
23                 endcase
24         end
25
26 endmodule
```

APÊNDICE N – data_RAM.sv

Código do arquivo data_RAM.sv

```
1  module data_RAM #(
2          parameter ADDR_WIDTH=10
3  )(
4          input logic [31:0] data,
5          input logic [(ADDR_WIDTH-1):0] read_addr, write_addr,
6
7          input read_clk, write_clk,
8          input memwrite, memread,
9
10         output logic [31:0] mem_out
11 );
12
13         // Declare the RAM variable
14         reg [31:0] ram[2**ADDR_WIDTH-1:0];
15
16         always_ff @ (posedge write_clk) begin
17                 // Write
18                 if (memwrite)
19                         ram[write_addr] <= data;
20         end
21
22         always_ff @ (posedge read_clk) begin
23                 // Read
24                 if(memread)
25                         mem_out <= ram[read_addr];
26         end
27
28 endmodule
```

APÊNDICE O – mux_write_back.sv

Código do arquivo mux_write_back.sv

```
1  module mux_write_back (
2      input logic [2:0] write_back,
3      input logic [31:0] mem_data,
4      input logic [31:0] alu_data,
5      input logic [31:0] pc_plus_one,
6      input logic [31:0] input_data,
7      input logic [31:0] rs_data,
8      output logic [31:0] mux_out
9  );
10
11     always_comb begin
12         case (write_back)
13             3'b000: mux_out = mem_data;
14             3'b001: mux_out = alu_data;
15             3'b010: mux_out = pc_plus_one;
16             3'b011: mux_out = input_data;
17             3'b100: mux_out = rs_data;
18                         default: mux_out = 32'b0;
19         endcase
20     end
21
22 endmodule
```

APÊNDICE P – clock_divider.sv

Código do arquivo clock_divider.sv

```
1  // fpga4student.com: FPGA projects, VHDL projects, Verilog
       projects
2  // Verilog project: Verilog code for clock divider on FPGA
3  // Top level Verilog code for clock divider on FPGA
4  module clock_divider #(
5      parameter DIVISOR = 28'd5_000_000
6  ) (
7      input logic clk_in,
8      output logic clk_out
9  );
10     logic [27:0] counter=28'd0;
11 // The frequency of the output clk_out
12 //  = The frequency of the input clk_in divided by DIVISOR
13 // For example: Fclk_in = 50Mhz, if you want to get 1Hz signal to
       blink LEDs
14 // You will modify the DIVISOR parameter value to 28'd50.000.000
15 // Then the frequency of the output clk_out = 50Mhz/50.000.000 =
       1Hz
16     always_ff @(posedge clk_in) begin
17         counter <= counter + 28'd1;
18         if(counter >= (DIVISOR - 1))
19             counter <= 28'd0;
20         clk_out <= (counter < DIVISOR/2) ? 1'b1:1'b0;
21     end
22
23 endmodule
```

APÊNDICE Q – button_debouncer.sv

Código do arquivo button_debouncer.sv

```
1  //fpga4student.com: FPGA projects, Verilog projects, VHDL
       projects
2  // Verilog code for button debouncing on FPGA
3  // debouncing module without creating another clock domain
4  // by using clock enable signal
5  module button_debouncer(
6      input logic btn_1, clk,
7      output logic btn_out
8  );
9
10     logic slow_clk_en;
11     logic Q1, Q2, Q2_bar, Q0;
12     clock_enable u1(clk,slow_clk_en);
13     debounce_dff_en d0(clk,slow_clk_en,btn_1,Q0);
14
15     debounce_dff_en d1(clk,slow_clk_en,Q0,Q1);
16     debounce_dff_en d2(clk,slow_clk_en,Q1,Q2);
17     assign Q2_bar = ~Q2;
18     assign btn_out = Q1 & Q2_bar;
19 endmodule
```

APÊNDICE R – clock_enable.sv

Código do arquivo clock_enable.sv

```
1  // Slow clock enable for debouncing button
2  module clock_enable(
3      input logic clk_50MHz,
4      output logic slow_clk_en
5  );
6      logic [25:0]   counter=0;
7      always_ff @(posedge clk_50MHz) begin
8          counter <= (counter>=124999)?0:counter+1;
9      end
10     assign slow_clk_en = (counter == 124999)?1'b1:1'b0;
11 endmodule
```

APÊNDICE S – debounce_dff_en.sv

Código do arquivo debounce_dff_en.sv

```
1     // D-flip-flop with clock enable signal for debouncing module
2 module debounce_dff_en(
3     input logic DFF_CLOCK, clock_enable, D,
4     output logic Q = 0
5 );
6     always_ff @(posedge DFF_CLOCK) begin
7         if(clock_enable == 1)
8             Q <= D;
9     end
10 endmodule
```

APÊNDICE T – display_output.sv

Código do arquivo display_output.sv

```
1  module display_output(
2      input logic flag_out, flag_in, flag_negative,
3      input logic [19:0] bin_value,
4      output logic [0:6] HEX6,
5      output logic [0:6] HEX5,
6      output logic [0:6] HEX4,
7      output logic [0:6] HEX3,
8      output logic [0:6] HEX2,
9      output logic [0:6] HEX1,
10     output logic [0:6] HEX0
11 );
12     logic [19:0] bin_value_abs;
13     logic [3:0] hundred_thousands, ten_thousands, thousands;
14     logic [3:0] hundreds, tens, units;
15
16     always_comb begin
17         if (flag_negative && flag_out) //complemento de dois
               apenas para out (em in, os switches s o em m dulo)
18             bin_value_abs = (~bin_value) + 20'd1;
19         else
20             bin_value_abs = bin_value;
21     end
22
23     bin_to_bcd bcd_converter(
24         .bin(bin_value_abs),
25         .hundred_thousands(hundred_thousands),
26         .ten_thousands(ten_thousands),
27         .thousands(thousands),
28         .hundreds(hundreds),
29         .tens(tens),
30         .units(units)
31     );
32
33     always_comb begin
34         HEX6 = 7'b1111110;
35         HEX5 = 7'b1111110;
```

```
36          HEX4 = 7'b1111110;
37          HEX3 = 7'b1111110;
38          HEX2 = 7'b1111110;
39          HEX1 = 7'b1111110;
40          HEX0 = 7'b1111110;
41
42          if (flag_out || flag_in) begin
43              HEX6 = (flag_negative) ? 7'b1111110 : 7'b1111111; //
                    Display "-" caso o n mero seja negativo
44              HEX5 = bcd_to_7seg(hundred_thousands);
45              HEX4 = bcd_to_7seg(ten_thousands);
46              HEX3 = bcd_to_7seg(thousands);
47              HEX2 = bcd_to_7seg(hundreds);
48              HEX1 = bcd_to_7seg(tens);
49              HEX0 = bcd_to_7seg(units);
50          end
51      end
52
53
54      function logic [6:0] bcd_to_7seg(input logic [3:0] digit);
55          case (digit)
56              4'd0: bcd_to_7seg = 7'b0000001;
57              4'd1: bcd_to_7seg = 7'b1001111;
58              4'd2: bcd_to_7seg = 7'b0010010;
59              4'd3: bcd_to_7seg = 7'b0000110;
60              4'd4: bcd_to_7seg = 7'b1001100;
61              4'd5: bcd_to_7seg = 7'b0100100;
62              4'd6: bcd_to_7seg = 7'b0100000;
63              4'd7: bcd_to_7seg = 7'b0001111;
64              4'd8: bcd_to_7seg = 7'b0000000;
65              4'd9: bcd_to_7seg = 7'b0000100;
66              default: bcd_to_7seg = 7'b1111111;
67          endcase
68      endfunction
69
70  endmodule
```

APÊNDICE U – bin_to_bcd.sv

Código do arquivo bin_to_bcd.sv

```
1  module bin_to_bcd (
2      input logic [19:0] bin,
3      output logic [3:0] hundred_thousands,
4      output logic [3:0] ten_thousands,
5      output logic [3:0] thousands,
6      output logic [3:0] hundreds,
7      output logic [3:0] tens,
8      output logic [3:0] units
9  );
10     integer i;
11     always_comb begin
12         hundred_thousands = 4'd0;
13         ten_thousands = 4'd0;
14         thousands = 4'd0;
15         hundreds = 4'd0;
16         tens = 4'd0;
17         units = 4'd0;
18
19         //Double Dabble Algorithm
20         for(i=19; i>=0; i=i-1) begin
21             if(hundred_thousands >= 5)
22                 hundred_thousands = hundred_thousands + 4'd3;
23             if(ten_thousands >= 5)
24                 ten_thousands = ten_thousands + 4'd3;
25             if(thousands >= 5)
26                 thousands = thousands + 4'd3;
27             if(hundreds >= 5)
28                 hundreds = hundreds + 4'd3;
29             if(tens >= 5)
30                 tens = tens + 4'd3;
31             if(units >= 5)
32                 units = units + 4'd3;
33
34             hundred_thousands = hundred_thousands << 1;
35             hundred_thousands[0] = ten_thousands[3];
36             ten_thousands = ten_thousands << 1;
```

```
37              ten_thousands[0] = thousands[3];
38              thousands = thousands << 1;
39              thousands[0] = hundreds[3];
40              hundreds = hundreds << 1;
41              hundreds[0] = tens[3];
42              tens = tens << 1;
43              tens[0] = units[3];
44              units = units << 1;
45              units[0] = bin[i];
46          end
47      end
48
49  endmodule
```